Cours Java & JavaFX

BTS SIO SLAM 2ème année

Version 1.0

Sébastien Inion

8 mars 2026

Licence

Licence Creative Commons Attribution - Partage dans les Mêmes Conditions 4.0 International (CC BY-SA 4.0)

© 2026 Sébastien Inion

Ce document est mis à disposition selon les termes de la licence Creative Commons Attribution - Partage dans les Mêmes Conditions 4.0 International (CC BY-SA 4.0).

Vous êtes autorisé à :

— partager : copier, distribuer et communiquer le document par tous moyens et sous tous formats ;

— adapter : remixer, transformer et créer à partir du document.

Selon les conditions suivantes :

— Attribution : vous devez créditer l'auteur, fournir un lien vers la licence et indiquer si des modifications ont été effectuées ;

— Partage dans les mêmes conditions : si vous modifiez ou transformez ce document, vous devez diffuser votre contribution sous la même licence.

Aucune restriction supplémentaire ne peut être appliquée légalement à l'utilisation de ce document au-delà des termes de la licence.

Pour consulter le texte complet de la licence :

https://creativecommons.org/licenses/by-sa/4.0/

Table des matières

Licence **3**

Présentation du module **11**

Mot de l'auteur 13

1 Origine et évolution du langage Java **15**

1.1 Introduction 15

1.2 Le contexte informatique des années 1990 15

1.3 Le projet Green 16

1.4 Le langage Oak 16

1.5 La popularisation de Java 17

1.6 Write Once, Run Anywhere 17

1.7 Évolution de Java 17

1.8 Java aujourd'hui 18

1.9 Les versions de Java 18

1.9.1 Les versions LTS 18

1.9.2 La version actuelle de Java 19

1.9.3 Quelques évolutions importantes du langage 19

1.9.4 Quelle version utiliser ? 19

2 L'environnement Eclipse **21**

2.1 Qu'est-ce qu'un IDE ? 21

2.1.1 Fonctionnalités principales d'un IDE 21

2.1.2 Exemple : Eclipse 21

2.2 Télécharger et installer Eclipse 22

2.2.1 Pré-requis 22

2.2.2 Principe de l'installation 24

2.2.3 Premier lancement 24

2.3 Le workspace 24

2.4 Création d'un projet Java 25

2.5 Structure d'un projet 25

2.6 Créer une classe 25
2.7 Premier programme 26
2.8 Exécuter un programme 26
2.9 La console 26
2.10 Les erreurs 26
2.11 Autocomplétion 27
2.12 Organisation des projets 27
2.13 Bonnes pratiques 27
2.14 Exercices d'application 28

3 Rigueur et organisation 29
3.1 Passer du code scolaire au code professionnel 29
3.2 Structure d'un projet Java 29
3.2.1 Isolation et visibilité 30
3.2.2 Contrôle de la visibilité 30
3.3 Entité métier 31
3.4 Principe de responsabilité unique 32
3.5 Encapsulation 33
3.6 Conventions de nommage 33
3.7 Lisibilité du code 34
3.8 Documenter son code avec JavaDoc 34
3.8.1 Exemple simple 35
3.8.2 Balises principales 36
3.8.3 Aide contextuelle dans les éditeurs 39
3.8.4 Exemple d'intérêt dans un IDE 39
3.8.5 Bonnes pratiques 40
3.8.6 À retenir 40
3.9 Contrôle de version 40
3.10 Exercices d'application 41

4 Structures de contrôle 45
4.1 Les structures conditionnelles 45
4.1.1 Structure if...else 45
4.1.2 La structure switch 46
4.2 Les structures itératives 48
4.2.1 La boucle while 48
4.2.2 La boucle do...while 48
4.2.3 La boucle for 49
4.2.4 Boucles sur séquences prédéfinies 49
4.3 Instructions break et continue 49

4.4 Attention aux boucles infinies 50
4.5 Exercices d'application 52

5 Méthodes et structuration du code 55
5.1 Méthodes de classe et programmation procédurale 55
5.2 Pourquoi utiliser des méthodes ? 56
5.3 Déclaration d'une méthode 56
5.4 Paramètres et valeur de retour 56
5.5 Effets de bord et fonctions pures 58
5.6 Visibilité des méthodes : public et private 59
5.7 Validation des paramètres d'une méthode 60
5.8 Factorisation d'un programme 61
5.9 Organisation logique d'un programme 62
5.10 Exemple complet structuré 62
5.11 Exercices d'application 64

6 Modélisation orientée objet 65
6.1 Du procédural à l'orienté objet 65
6.2 Définir une classe 66
6.3 Le mot-clé `private` et le principe d'encapsulation 67
6.4 Instanciation d'un objet 68
6.5 Getters et setters 69
6.6 Le constructeur 69
6.7 La référence `this` 71
6.7.1 Pourquoi a-t-on besoin de `this` ? 71
6.8 Exercices d'application 74

7 Architecture applicative et séparation des couches 77
7.1 Pourquoi parler d'architecture ? 77
7.2 Le problème d'un code mal structuré 77
7.3 Principe fondamental : la séparation des responsabilités 78
7.4 Le modèle MVC 78
7.5 Exemple cohérent d'architecture MVC 79
7.6 Pourquoi cette séparation est essentielle ? 81
7.7 Préparation à JavaFX 81
7.8 Exercices d'application 82

8 Héritage et polymorphisme 85
8.1 Pourquoi l'héritage ? 85
8.2 Relation **est-un** et **a-un** 85

8.3 Définir une classe mère 86
8.4 Créer une sous-classe 86
8.5 Polymorphisme 88
8.6 Classes abstraites 88
8.7 Principe de substitution de Liskov 89
8.8 Bonnes pratiques 90
8.9 Exercices d'application 92

9 Interfaces, composition 95
9.1 Pourquoi les interfaces ? 95
9.2 Définition d'une interface 97
9.3 Implémentation d'une interface 98
9.4 Introduction à la composition 99
9.5 Favoriser la composition 99
9.6 Le pattern Strategy 101
9.6.1 Principe 101
9.6.2 Structure 101
9.6.3 Exemple 102
9.6.4 Avantages 103
9.6.5 Lien avec les interfaces 103
9.7 Introduction aux principes SOLID 103

10 Collections et généricité 111
10.1 Limites des tableaux 111
10.2 Le framework de collections 113
10.3 La collection List 113
10.4 Parcourir une collection 114
10.5 Collections d'objets 114
10.6 Pourquoi utiliser des génériques ? 115
10.7 Avantages des génériques 115
10.8 Set 115
10.9 Intérêt des collections 119
10.10 Comparer et trier des objets 121
10.10.1 Ordre naturel avec `Comparable` 124
10.10.2 Ordres multiples avec `Comparator` 124
10.10.3 Lien avec les collections triées 125

11 Gestion des exceptions et robustesse 129
11.1 Qu'est-ce qu'une exception ? 129
11.2 Le bloc try / catch 129

11.3 Plusieurs blocs catch . . . 130
11.4 Le bloc finally . . . 130
11.5 Le mot-clé throw . . . 131
11.6 Exceptions vérifiées et non vérifiées . . . 131
11.7 Créer sa propre exception . . . 131
11.8 Bonnes pratiques . . . 132
11.9 Exercices d'application . . . 133

12 Entrées / Sorties et communication 135
12.1 Introduction . . . 135
12.2 Lecture et écriture de fichiers . . . 135
12.3 Format CSV . . . 136
12.4 Bases de données . . . 137
12.4.1 Ajout du driver JDBC MySQL dans Eclipse . . . 139
12.5 Les API . . . 140
12.6 Comparaison des différentes approches . . . 142
12.7 Exercices d'application . . . 143

13 Expressions Lambda 147
13.1 Objectifs . . . 147
13.2 Motivation . . . 147
13.3 Principe d'une expression lambda . . . 148
13.4 Interfaces fonctionnelles . . . 148
13.5 Premier exemple avec lambda . . . 149
13.6 Lambda avec bloc d'instructions . . . 149
13.7 Exemples simples . . . 149
13.7.1 Lambda sans paramètre . . . 149
13.7.2 Lambda avec un paramètre . . . 150
13.7.3 Lambda avec plusieurs paramètres . . . 150
13.8 Utilisation avec les collections . . . 150
13.9 Références de méthodes . . . 150
13.10 Utilisation dans les événements . . . 151
13.11 Bonnes pratiques . . . 151
13.12 Exercices . . . 151
13.12.1 Exercice 1 . . . 151
13.12.2 Exercice 2 . . . 151
13.12.3 Exercice 3 . . . 152
13.13 À retenir . . . 152

14 Introduction à JavaFX 153
14.1 Objectifs . . . 153
14.2 Applications console et interfaces graphiques . . . 153
14.3 Qu'est-ce que JavaFX ? . . . 154
14.4 Architecture d'une application JavaFX . . . 154
14.4.1 Le Stage . . . 154
14.4.2 La Scene . . . 154
14.4.3 Les Nodes . . . 154
14.5 Structure d'une application JavaFX . . . 155
14.6 Premier programme JavaFX . . . 155
14.7 Explication du programme . . . 155
14.7.1 Héritage de la classe Application . . . 155
14.7.2 La méthode start . . . 156
14.7.3 Afficher la fenêtre . . . 156
14.8 Ajouter du contenu à la fenêtre . . . 156
14.9 Analyse du programme . . . 157
14.9.1 Création d'un composant . . . 157
14.9.2 Création de la scène . . . 157
14.9.3 Association de la scène à la fenêtre . . . 157
14.10 Cycle de vie d'une application JavaFX . . . 157
14.11 Bonnes pratiques . . . 158
14.12 Exercices . . . 158
14.12.1 Exercice 1 . . . 158
14.12.2 Exercice 2 . . . 158
14.12.3 Exercice 3 . . . 158

15 Tests unitaires avec JUnit 159
15.1 Pourquoi tester un programme . . . 159
15.2 Principe d'un test unitaire . . . 159
15.3 La bibliothèque JUnit . . . 160
15.4 Les assertions . . . 160
15.5 Tester une classe . . . 161
15.6 Initialisation et nettoyage des tests . . . 162
15.6.1 Méthodes principales . . . 162
15.6.2 Exemple complet . . . 163
15.7 Tester les exceptions . . . 164
15.8 Organisation des tests . . . 165
15.9 Utilisation de JUnit dans Eclipse . . . 165
15.10 Conclusion . . . 166

Présentation du support de cours

Ce module a pour objectif de former les étudiants de BTS SIO SLAM à la conception et au développement d'une application métier complète en Java, incluant une interface graphique professionnelle en JavaFX.

Prérequis :

Ce cours part du principe que l'étudiant :

— maîtrise les bases de la programmation orientée objet (classes, objets, encapsulation, héritage simple),

— dispose d'un environnement de développement Eclipse opérationnel,

— sait créer, compiler et exécuter un projet Java,

— a déjà réalisé un premier programme de type *Hello World*.

L'objectif de ce module n'est donc pas de réapprendre les bases, mais de passer à un niveau supérieur de rigueur, d'architecture et de professionnalisation.

Ce module marque le passage d'un code scolaire à une approche de développement professionnelle.

La progression pédagogique est structurée autour des chapitres suivants :

1. Rigueur et organisation d'un projet Java
2. Maîtrise avancée des fondamentaux
3. Méthodes et structuration du code
4. Modélisation orientée objet
5. Architecture applicative et MVC
6. Héritage et polymorphisme
7. Interfaces, abstraction et principes SOLID
8. Collections et généricité
9. Exceptions et robustesse
10. Tests unitaires avec JUnit
11. Introduction à JavaFX
12. Architecture JavaFX professionnelle
13. Composants avancés et dynamique

14. Projet final : application métier complète en JavaFX

Cette progression suit une logique volontairement progressive et professionnelle.

— Les chapitres 1 à 3 consolident la rigueur de développement et la structuration du code.
— Les chapitres 4 à 7 construisent une maîtrise solide de la programmation orientée objet et des principes d'architecture.
— Les chapitres 8 et 9 renforcent la manipulation de données dynamiques et la robustesse applicative.
— Le chapitre 10 introduit la culture du test unitaire, indispensable dans un contexte professionnel.
— Les chapitres 11 à 13 permettent l'apprentissage progressif de JavaFX et de l'architecture MVC graphique.
— Le chapitre 14 aboutit à la réalisation d'une application métier complète intégrant l'ensemble des compétences acquises.

L'objectif est de construire d'abord une couche métier solide et testée, avant d'y connecter une interface graphique professionnelle.

Cette approche reflète les pratiques réelles du développement en entreprise.

Chaque chapitre se termine par une série d'exercices destinés à mettre en application les notions étudiées.

Il est essentiel de les réaliser sérieusement et de chercher activement à résoudre les problèmes posés. La compréhension ne vient pas de la lecture de la solution, mais de l'effort intellectuel fourni pour y parvenir.

Les corrections sont disponibles dans le dépôt GitHub suivant :

https://github.com/SebInfo/CorrectionJavaExercices

En effet, consulter la correction sans avoir tenté l'exercice avec rigueur et persévérance serait comparable à lire un roman policier en connaissant déjà la fin.

Comme je dis toujours : « C'est en sciant du bois que Léonard devint scie ! Et c'est en programmant qu'on devient programmeur... ;) ».

Mot de l'auteur

J'espère que ce support de cours vous apportera non seulement des connaissances techniques, mais également une méthode de travail rigoureuse et structurée.

Apprendre à programmer ne consiste pas uniquement à écrire du code, mais à apprendre à raisonner, à organiser sa pensée et à construire des solutions solides.

La programmation est un métier exigeant, mais également passionnant. Java est un langage riche, puissant et formateur. Il constitue un excellent outil pour développer des compétences durables.

Je vous souhaite de trouver, au fil de ces chapitres, à la fois méthode, progression et plaisir dans l'apprentissage de ce beau langage.

À l'ère de l'intelligence artificielle, des outils comme *ChatGPT* ou *Claude* peuvent constituer une aide précieuse. Ils permettent d'explorer des pistes, de clarifier des notions ou d'obtenir des exemples supplémentaires.

Cependant, ces outils doivent rester des supports à la réflexion. Ils ne remplacent ni l'effort personnel, ni la compréhension profonde, ni la créativité du développeur.

L'intelligence artificielle assiste la pensée, mais ne s'y substitue pas. C'est votre capacité à analyser, concevoir et structurer des solutions qui fera de vous un professionnel compétent dans ce métier exigeant et passionnant.

Cordialement,
Sébastien Inion

Chapitre 1

Origine et évolution du langage Java

1.1 Introduction

Avant d'apprendre à programmer en Java, il est intéressant de comprendre l'origine de ce langage et les raisons qui ont conduit à sa création.

Java est aujourd'hui l'un des langages de programmation les plus utilisés au monde. Il est employé dans de nombreux domaines :

- applications d'entreprise
- les applications Android (Java a longtemps été le langage principal, aujourd'hui Kotlin est recommandé par Google)
- les systèmes distribués
- serveurs web
- logiciels scientifiques
- systèmes embarqués

Mais ce langage n'a pas été créé pour ces usages. Son histoire remonte au début des années 1990.

1.2 Le contexte informatique des années 1990

Au début des années 1990, l'informatique évolue rapidement.

Les ordinateurs personnels deviennent de plus en plus répandus et les réseaux commencent à se développer. Cependant, les logiciels restent souvent dépendants du système d'exploitation et de l'architecture matérielle sur lesquels ils ont été développés.

Un programme compilé pour une plateforme particulière ne fonctionne généralement pas sur une autre sans être recompilé.

Par exemple :

- un programme compilé pour Windows ne fonctionne pas directement sous Unix
- un programme compilé pour une architecture matérielle donnée ne fonctionne pas sur une autre

Certains langages comme le **C** avaient déjà été conçus pour être relativement portables. Le code source pouvait être recompilé sur différentes machines, ce qui facilitait le portage des logiciels.

Cependant, cette portabilité restait limitée : il fallait souvent adapter le code au système cible et le recompiler pour chaque plateforme.

L'objectif poursuivi par les concepteurs de Java était d'aller plus loin en proposant un langage permettant d'exécuter le même programme sur différentes machines **sans recompilation**.

1.3 Le projet Green

Java est né chez l'entreprise **Sun Microsystems** au début des années 1990.

Une équipe dirigée par l'ingénieur **James Gosling** travaille sur un projet appelé **Green Project**.

L'objectif initial n'était pas de créer un langage pour les ordinateurs personnels, mais pour les **appareils électroniques grand public**.

L'idée était de développer un langage capable de fonctionner sur différents types d'appareils :

— télévisions interactives
— décodeurs
— systèmes embarqués

Pour cela, il fallait un langage :

— portable
— simple
— sûr
— indépendant du matériel

1.4 Le langage Oak

Le langage créé par l'équipe de Sun Microsystems s'appelait initialement **Oak**.

Ce langage possédait déjà plusieurs caractéristiques qui feront le succès de Java :

— programmation orientée objet
— gestion automatique de la mémoire
— indépendance vis-à-vis du système d'exploitation

Cependant, le nom Oak était déjà utilisé pour une autre technologie. Le langage fut donc renommé **Java** en 1995.

1.5 La popularisation de Java

La véritable popularisation de Java arrive avec l'explosion du **World Wide Web** au milieu des années 1990.

Java permet alors d'exécuter des petits programmes appelés **applets** directement dans les navigateurs web.

Même si les applets ont aujourd'hui disparu, elles ont largement contribué à faire connaître le langage.

Java devient rapidement populaire grâce à plusieurs caractéristiques importantes :

— portabilité
— sécurité
— simplicité relative
— forte bibliothèque standard

1.6 Write Once, Run Anywhere

L'une des idées fondamentales de Java est résumée par la phrase :

Write Once, Run Anywhere

Cela signifie qu'un programme Java peut être exécuté sur différentes plateformes sans modification.

Cela est rendu possible grâce à la **Java Virtual Machine (JVM)**.

Le code Java est d'abord compilé en un langage intermédiaire appelé **bytecode**. Ce bytecode est ensuite exécuté par la JVM.

Chaque système d'exploitation possède sa propre JVM.

1.7 Évolution de Java

Depuis sa création en 1995, Java a connu de nombreuses évolutions.

De nouvelles versions ont introduit des fonctionnalités importantes :

— amélioration des performances
— nouvelles bibliothèques
— expressions lambda
— API modernes

Aujourd'hui, Java est maintenu principalement par l'entreprise **Oracle** qui a racheté Sun Microsystems en 2010.

1.8 Java aujourd'hui

Java reste aujourd'hui l'un des langages les plus utilisés dans l'industrie.

Il est notamment utilisé pour :

— les applications d'entreprise
— les serveurs web
— les applications Android
— les systèmes distribués
— les outils professionnels

Sa stabilité, sa portabilité et la richesse de son écosystème expliquent sa longévité.

1.9 Les versions de Java

Depuis sa première version publiée en 1995, Java a connu de nombreuses évolutions. Au fil du temps, de nouvelles versions du langage et de la plateforme Java ont été introduites afin d'améliorer :

— les performances
— la sécurité
— les bibliothèques standard
— les fonctionnalités du langage

Pendant longtemps, les nouvelles versions de Java étaient publiées de manière irrégulière. Depuis 2017, le projet Java suit un **cycle de publication rapide** : une nouvelle version est publiée environ tous les **six mois**.

Ces versions apportent généralement des améliorations progressives du langage et de la machine virtuelle Java.

Cependant, toutes les versions ne sont pas utilisées de la même manière dans les environnements professionnels.

1.9.1 Les versions LTS

Certaines versions de Java sont appelées **LTS** (Long Term Support).

Une version LTS bénéficie d'un support sur une longue durée. Cela signifie que ces versions reçoivent pendant plusieurs années :

— des corrections de bugs
— des mises à jour de sécurité
— des améliorations de stabilité

Ces versions sont privilégiées par les entreprises car elles offrent une plus grande stabilité pour les applications en production.

Les principales versions LTS publiées ces dernières années sont :

Version	Année
Java 8	2014
Java 11	2018
Java 17	2021
Java 21	2023
Java 25	2025

1.9.2 La version actuelle de Java

Au moment de la rédaction de ce cours, la version la plus récente de Java est **Java 25**, publiée en 2025. Il s'agit également d'une version **LTS**.

Cette version apporte notamment :

— des améliorations de performance de la machine virtuelle Java (JVM)

— de nouvelles fonctionnalités du langage

— des améliorations des bibliothèques standard

Les nouvelles versions de Java continuent d'améliorer la performance, la sécurité et la facilité de développement.

1.9.3 Quelques évolutions importantes du langage

Certaines versions de Java ont introduit des fonctionnalités particulièrement importantes :

— **Java 5** : introduction des **génériques**, des **annotations** et de la boucle `for-each`

— **Java 8** : introduction des **expressions lambda** et de l'**API Stream**

— **Java 9** : introduction du **système de modules**

— **Java 17** : amélioration du langage et stabilisation de nombreuses fonctionnalités récentes

— **Java 21** : introduction des **virtual threads** permettant une meilleure gestion de la concurrence

Ces évolutions permettent à Java de rester un langage moderne tout en conservant une très forte compatibilité avec les versions précédentes.

1.9.4 Quelle version utiliser ?

Dans ce cours, nous utiliserons une version récente du **JDK** compatible avec les standards actuels du langage.

Dans le monde professionnel, les entreprises utilisent majoritairement les versions **LTS**, car elles garantissent la stabilité et la maintenance des applications sur le long terme.

Aujourd'hui, les versions les plus utilisées sont généralement :

— **Java 17**

— **Java 21**

— et progressivement **Java 25**

Conclusion

Java est né au début des années 1990 dans un contexte où la portabilité des logiciels était un problème majeur.

Grâce à l'introduction de la machine virtuelle Java et à une conception orientée objet robuste, le langage s'est rapidement imposé comme une technologie majeure.

Comprendre l'origine de Java permet de mieux saisir certains choix de conception du langage que nous découvrirons dans les chapitres suivants.

Chapitre 2

L'environnement Eclipse

2.1 Qu'est-ce qu'un IDE ?

Un **IDE** (Integrated Development Environment), ou **environnement de développement intégré**, est un logiciel qui regroupe dans une seule application les principaux outils nécessaires pour développer un programme informatique.

Sans IDE, un développeur devrait utiliser plusieurs outils séparés :

— un éditeur de texte pour écrire le code
— un compilateur pour transformer le code source en programme exécutable
— un outil pour lancer le programme
— des outils pour analyser et corriger les erreurs

Un IDE regroupe tous ces outils dans une interface unique afin de faciliter le travail du développeur.

2.1.1 Fonctionnalités principales d'un IDE

Un IDE propose généralement les fonctionnalités suivantes :

— **éditeur de code** avec coloration syntaxique
— **autocomplétion** du code
— **gestion des projets**
— **compilation automatique**
— **exécution des programmes**
— **outil de débogage** permettant d'analyser l'exécution d'un programme

Ces outils permettent de développer des applications plus rapidement et de limiter les erreurs.

2.1.2 Exemple : Eclipse

Dans ce cours nous utiliserons l'IDE **Eclipse** pour programmer en Java.

Eclipse permet notamment de :

— créer et organiser des projets Java
— écrire du code avec assistance (autocomplétion)
— compiler automatiquement les programmes [1]
— exécuter les applications
— analyser et corriger les erreurs

Il existe d'autres IDE pour le développement Java comme **IntelliJ IDEA** ou **NetBeans**, mais Eclipse est très répandu dans l'enseignement et dans de nombreux environnements professionnels.

L'utilisation d'un IDE est aujourd'hui indispensable pour développer des applications de taille importante.

2.2 Télécharger et installer Eclipse

Eclipse est un logiciel libre et gratuit. Il peut être téléchargé sur le site officiel du projet Eclipse :

https://www.eclipse.org/downloads/

Plusieurs versions d'Eclipse existent selon les langages utilisés. Pour le développement Java, il est recommandé de télécharger la version :

Eclipse IDE for Java Developers

2.2.1 Pré-requis

Avant d'installer Eclipse, il est nécessaire d'avoir installé **Java** sur son ordinateur. Plus précisément, il faut disposer d'un **JDK (Java Development Kit)**.

Choisir la version du JDK

Plusieurs versions du JDK sont disponibles. Comme expliqué dans le chapitre précédent, Java publie régulièrement de nouvelles versions.

Pour le développement et l'apprentissage du langage, il est recommandé d'utiliser une version **LTS (Long Term Support)**. Ces versions sont maintenues pendant plusieurs années et sont largement utilisées dans les environnements professionnels.

Au moment de la rédaction de ce cours, la version LTS la plus récente est **Java 25**.

Il est donc conseillé d'installer :

1. La compilation est l'opération qui consiste à transformer le code source écrit par le programmeur en un langage compréhensible par la machine. Dans un langage compilé, cette transformation est réalisée avant l'exécution du programme. À l'inverse, dans un langage interprété, le programme est analysé et exécuté instruction par instruction par un interpréteur. Java utilise un modèle hybride : le code source est d'abord compilé en *bytecode*, puis ce bytecode est exécuté par la machine virtuelle Java (JVM).

JDK 25

Cette version est stable, maintenue sur le long terme et compatible avec les outils utilisés dans ce cours, notamment l'IDE Eclipse.

Le JDK peut être téléchargé sur les sites suivants :

— Site officiel Oracle

— Projet OpenJDK

Explication simple

Le **JDK** est l'ensemble des outils nécessaires pour développer des programmes en Java.

Lorsque nous écrivons un programme Java, nous produisons un fichier texte contenant du code appelé **code source**. Ce code ne peut pas être exécuté directement par l'ordinateur.

Le JDK fournit les outils permettant de :

— **compiler** le programme Java

— **exécuter** le programme

— utiliser les **bibliothèques standard** du langage Java

Sans JDK, il est donc impossible de développer ou d'exécuter des programmes Java.

Explication plus technique

Le **Java Development Kit (JDK)** est l'environnement complet de développement Java fourni par Oracle ou par des distributions compatibles comme **OpenJDK**.

Il contient notamment :

— le **compilateur Java** (`javac`) qui transforme le code source `.java` en **bytecode** `.class`

— la **Java Virtual Machine (JVM)** qui permet d'exécuter le bytecode

— les **bibliothèques standard Java** utilisées par les programmes

— différents **outils de développement** (documentation, gestion d'archives, débogage, etc.)

Le fonctionnement général d'un programme Java est le suivant :

Code source Java (.java)

↓

Compilation avec le compilateur `javac`

↓

Bytecode (.class)

↓

Exécution par la **JVM**

Cette architecture permet à Java d'être **portable** : un même programme compilé peut être exécuté sur différents systèmes d'exploitation tant qu'une JVM compatible est disponible.

Une version récente du JDK peut être téléchargée sur le site officiel d'Oracle ou sur une distribution libre comme **OpenJDK**.

Dans mon environnement de travail, Eclipse utilise la distribution **Temurin** du JDK. La version installée est **Java 21 LTS**.

Cependant, la plupart des exemples et exercices présentés dans ce livre utilisent uniquement les fonctionnalités fondamentales du langage Java. Ils restent donc compatibles avec des versions plus anciennes du langage, comme **Java 8**, qui est encore largement utilisée dans certains environnements professionnels.

2.2.2 Principe de l'installation

L'installation d'Eclipse est relativement simple.

Les étapes principales sont :

1. télécharger l'installateur Eclipse
2. lancer l'installateur
3. choisir la version **Eclipse IDE for Java Developers**
4. choisir le dossier d'installation
5. lancer l'installation

Une fois l'installation terminée, Eclipse peut être lancé depuis le menu des applications.

2.2.3 Premier lancement

Lors du premier démarrage, Eclipse demande de choisir un **workspace**.

Le workspace est un dossier dans lequel Eclipse stocke :

— les projets
— les fichiers sources
— certains paramètres de l'environnement

Il est conseillé de choisir un dossier dédié aux projets Java.

2.3 Le workspace

Lors du premier lancement d'Eclipse, il est demandé de choisir un **workspace**.

Le workspace est un dossier dans lequel Eclipse stocke :

— les projets
— les paramètres de l'environnement
— les fichiers de configuration

Chaque étudiant peut utiliser un workspace personnel.

2.4 Création d'un projet Java

Pour créer un projet :

1. Menu **File**
2. **New**
3. **Java Project**

Donner un nom au projet :

`PremierProjet`

Puis cliquer sur **Finish**.

2.5 Structure d'un projet

Un projet Java contient généralement plusieurs dossiers :
— **src** : contient les fichiers source Java
— **JRE System Library** : contient les bibliothèques Java
Le dossier `src` contient les classes du programme.

2.6 Créer une classe

Pour créer une classe :

1. clic droit sur **src**
2. **New**
3. **Class**

Nom de la classe :

`Main`

Cocher l'option :

public static void main(String[] args)

Puis cliquer sur **Finish**.

2.7 Premier programme

Écrire le programme suivant :

```
public class Main {

    public static void main(String[] args) {

        System.out.println("Bonjour Java");

    }

}
```

2.8 Exécuter un programme

Pour exécuter le programme :
— clic droit sur la classe
— **Run As**
— **Java Application**
Le résultat apparaît dans la console d'Eclipse.

2.9 La console

La console affiche :
— les résultats du programme
— les messages d'erreur
— les traces d'exécution
Exemple :

```
Bonjour Java
```

2.10 Les erreurs

Eclipse signale les erreurs dans le code :
— un trait rouge sous le code
— une croix rouge dans la marge
En plaçant la souris sur l'erreur, Eclipse affiche une explication.

2.11 Autocomplétion

Eclipse propose automatiquement du code pendant la saisie.
Par exemple :

```
System.out.pr
```

Eclipse propose :

```
println()
```

L'autocomplétion permet :
— de coder plus rapidement
— d'éviter certaines erreurs

2.12 Organisation des projets

Un projet Java est généralement organisé en **packages**.
Les packages permettent de regrouper les classes par fonctionnalité.
Exemple :

```
com.monprojet

    model
    service
    controller
```

Nous utiliserons cette organisation dans les projets plus complexes.

2.13 Bonnes pratiques

Quelques bonnes pratiques lors de l'utilisation d'Eclipse :
— utiliser un nom clair pour les projets
— organiser les classes dans des packages
— corriger les erreurs dès qu'elles apparaissent
— sauvegarder régulièrement

2.14 Exercices d'application

Exercice 2.1

Créer un projet Java nommé :

`HelloJava`

Créer une classe `Main` qui affiche :

```
Bienvenue dans mon premier programme Java
```

Exercice 2.2

Modifier le programme pour afficher :
— votre nom
— votre prénom
— votre formation

Exercice 2.3

Créer un nouveau projet Java nommé : `Calcul`
Créer un programme qui affiche le résultat de : 5 + 3

Chapitre 3

Rigueur et organisation

3.1 Passer du code scolaire au code professionnel

Jusqu'à présent, les programmes réalisés avaient souvent un objectif simple : faire fonctionner un programme.

Dans un contexte professionnel, l'objectif est différent. Il ne s'agit plus seulement de produire un programme qui fonctionne, mais de concevoir une application :

— compréhensible
— maintenable
— évolutive

Un programme peut fonctionner parfaitement aujourd'hui et devenir très difficile à modifier quelques mois plus tard si son organisation est mauvaise.

La qualité d'un logiciel dépend donc autant de son **architecture** que de son fonctionnement.

Certaines notions présentées dans ce chapitre peuvent sembler abstraites. C'est normal : elles seront approfondies dans les chapitres suivants.

3.2 Structure d'un projet Java

Un projet Java professionnel n'est généralement pas constitué d'un unique fichier. Le code est organisé en **packages** afin de structurer l'application.

Exemple d'organisation :

```
bibliotheque-app/
  |-- src/
       |-- main/
            |-- java/
                 |-- com/
                      |-- btssio/
```

```
                        |-- bibliotheque/
                             |-- Main.java
                             |-- model/
                             |-- service/
                             |-- controller/
                             |-- repository/
          |-- resources/
```

Cette organisation permet de séparer les différentes parties de l'application.

Dans ce livre, les exemples resteront volontairement simples. Cependant, il est important de comprendre dès maintenant que les applications professionnelles sont toujours organisées en packages.

Rôle des principaux packages

— **model** : contient les classes représentant les données du métier
— **service** : contient la logique métier et les règles de gestion
— **controller** : gère les interactions entre l'utilisateur et l'application
— **repository** : gère l'accès aux données (base de données, fichiers)

Cette séparation rend le code plus clair et plus facile à maintenir.

3.2.1 Isolation et visibilité

L'utilisation de packages ne sert pas uniquement à organiser les fichiers. Elle permet également de contrôler la **visibilité** des classes et des méthodes.

En Java, tous les éléments d'un programme ne sont pas forcément accessibles depuis n'importe quelle partie du code. On peut limiter leur accès afin de protéger certaines parties de l'application.

Les packages permettent donc de créer une forme d'**isolation** entre les différentes parties du programme.

Par exemple :
— les classes du package `model` représentent les objets de l'application ;
— les classes du package `service` utilisent ces objets pour appliquer la logique métier ;
— les classes du package `controller` orchestrent le fonctionnement global de l'application.

Chaque partie du programme a donc un rôle précis.

3.2.2 Contrôle de la visibilité

Java propose plusieurs niveaux de visibilité pour les classes, attributs et méthodes :
— `public` : accessible depuis n'importe quel package ;

— `protected` : accessible dans le même package et dans les classes héritées ;
— visibilité par défaut (sans mot-clé) : accessible uniquement dans le même package ;
— `private` : accessible uniquement dans la classe.

Grâce aux packages, il est donc possible de :
— cacher certaines classes qui ne doivent pas être utilisées directement ;
— limiter l'accès à certaines méthodes internes ;
— exposer uniquement les classes nécessaires aux autres parties de l'application.

Cette organisation permet de mieux structurer les applications et de réduire les risques d'erreurs.

3.3 Entité métier

Une **entité métier** représente un objet du domaine fonctionnel de l'application.

Par exemple :
— un livre dans une bibliothèque
— un client dans une application de vente

Une entité métier contient généralement :
— des attributs (les données)
— un état
— des méthodes décrivant son comportement

Exemple

```java
public class Livre {

    private String titre;
    private String auteur;
    private boolean emprunte;

    public Livre(String titre, String auteur) {
        setTitre(titre);
        setAuteur(auteur);
        this.emprunte = false;
    }

    public void setTitre(String titre) {
        if (titre == null || titre.isBlank()) {
            throw new IllegalArgumentException("Titre invalide");
        }
        this.titre = titre;
    }
```

```java

    public void setAuteur(String auteur) {
        if (auteur == null || auteur.isBlank()) {
            throw new IllegalArgumentException("Auteur invalide")
                ;
        }
        this.auteur = auteur;
    }

    public void emprunter() {
        emprunte = true;
    }

    public void rendre() {
        emprunte = false;
    }
}
```

Dans cet exemple :

— les attributs sont `titre`, `auteur` et `emprunte`

— l'état du livre correspond à la valeur de `emprunte`

— les méthodes `emprunter()` et `rendre()` représentent les actions possibles sur le livre

3.4 Principe de responsabilité unique

Une bonne application est composée de classes ayant chacune un rôle clair.

Ce principe est appelé **principe de responsabilité unique**.

Règle importante

Une classe doit avoir une seule responsabilité.

Si une classe fait trop de choses différentes, elle devient difficile à comprendre et à modifier.

Exemple de mauvaise conception

```java
public class LivreManager {
    public void afficherLivre() { }
    public void sauvegarderLivre() { }
    public void calculerRetard() { }
}
```

Cette classe mélange plusieurs rôles :

— l'affichage
— la sauvegarde des données
— la logique métier

Organisation plus claire

Une organisation plus claire consiste à séparer ces responsabilités :

— **Livre** : représente les données du livre
— **LivreService** : contient les règles métier
— **LivreRepository** : gère la sauvegarde des données

Chaque classe possède ainsi un rôle précis.

3.5 Encapsulation

L'encapsulation consiste à protéger les données d'une classe.

Les attributs d'une classe doivent généralement être déclarés `private` afin d'empêcher leur modification directe.

```
public class Etudiant {
    private String nom;

    public String getNom() {
        return nom;
    }
}
```

Cette approche permet de contrôler la manière dont les données sont modifiées.

3.6 Conventions de nommage

Afin de rendre le code plus lisible, certaines conventions de nommage sont utilisées en Java.

Classes

Les noms de classes utilisent la convention **PascalCase**.

```
public class GestionEtudiant { }
```

Variables

Les variables utilisent la convention **camelCase**.

```
int nombreEtudiants;
double moyenneGenerale;
```

Constantes

Les constantes sont écrites en majuscules.

```
public static final int CAPACITE_MAX = 30;
```

3.7 Lisibilité du code

Un programme doit pouvoir être compris rapidement par un autre développeur.
Pour cela il est important de respecter certaines règles simples :

— utiliser des noms explicites
— écrire des méthodes courtes
— indenter correctement le code

Exemple

Code peu lisible

```
int a=5;
int b=6;
System.out.println(a+b);
```

Code plus clair

```
int premierNombre = 5;
int secondNombre = 6;

int somme = premierNombre + secondNombre;

System.out.println("Resultat : " + somme);
```

3.8 Documenter son code avec JavaDoc

Documenter son code est une bonne pratique essentielle en programmation. Un programme n'est pas seulement écrit pour être exécuté par une machine : il doit aussi pouvoir être relu, compris et maintenu par d'autres développeurs, ou par son propre auteur plusieurs semaines plus tard.

En Java, la documentation du code se fait généralement avec **JavaDoc**. JavaDoc est à la fois :

— une **convention d'écriture** de commentaires ;
— un **format standard** reconnu par les outils Java ;
— un **générateur de documentation** à partir du code source.

Autrement dit, JavaDoc permet d'écrire une documentation directement dans le code, puis de produire automatiquement une documentation technique lisible.

À quoi sert JavaDoc ?

JavaDoc présente plusieurs intérêts :

— expliquer le rôle d'une classe ou d'une méthode ;
— préciser les paramètres attendus ;
— indiquer la valeur retournée ;
— signaler les exceptions possibles ;
— faciliter la réutilisation du code ;
— générer automatiquement une documentation exploitable par d'autres développeurs.

JavaDoc est particulièrement utile lorsqu'un projet contient plusieurs classes, plusieurs développeurs, ou lorsqu'une méthode n'est pas immédiatement compréhensible.

Syntaxe générale

Un commentaire JavaDoc se place juste au-dessus de l'élément à documenter.

Sa syntaxe commence par :

```
/**
 * ...
 */
```

Il ne faut pas confondre JavaDoc avec les autres types de commentaires en Java :

— `// commentaire sur une ligne`
— `/* commentaire classique sur plusieurs lignes */`
— `/** commentaire JavaDoc */`

Seuls les commentaires écrits avec `/** ... */` sont interprétés comme de la documentation JavaDoc.

3.8.1 Exemple simple

```
1 /**
2  * Représente un étudiant.
3  */
```

```java
public class Etudiant {

    private String nom;

    /**
     * Retourne le nom de l'étudiant.
     * @return le nom de l'étudiant
     */
    public String getNom() {
        return nom;
    }
}
```

Dans cet exemple :

— la première JavaDoc documente la classe `Etudiant` ;

— la seconde documente la méthode `getNom()` ;

— la balise `@return` précise la valeur renvoyée par la méthode.

Exemple avec paramètres et exception

```java
/**
 * Modifie l'âge de l'étudiant.
 *
 * @param age le nouvel âge de l'étudiant
 * @throws IllegalArgumentException si l'âge est négatif
 */
public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("L'âge doit être
            positif");
    }
    this.age = age;
}
```

Cet exemple montre que JavaDoc permet de documenter :

— les paramètres avec `@param` ;

— les exceptions avec `@throws`.

3.8.2 Balises principales

Les balises les plus utiles sont les suivantes :

— `@param` : décrit un paramètre d'une méthode ;

- `@return` : décrit la valeur retournée ;
- `@throws` : indique une exception que la méthode peut lever ;
- `@author` : indique l'auteur ;
- `@version` : indique une version du code ;
- `@see` : permet de faire un renvoi vers une autre classe ou méthode.

Exemple plus complet

```java
/**
 * Représente un produit vendable.
 *
 * @author Nom Prenom
 * @version 1.0
 */
public class Produit {

    private String nom;
    private double prix;

    /**
     * Construit un produit à partir d'un nom et d'un prix.
     *
     * @param nom le nom du produit
     * @param prix le prix du produit
     * @throws IllegalArgumentException si le nom est vide
     *                                  ou si le prix est négatif
     */
    public Produit(String nom, double prix) {
        setNom(nom);
        setPrix(prix);
    }

    /**
     * Retourne le prix du produit.
     *
     * @return le prix du produit
     */
    public double getPrix() {
        return prix;
    }
}
```

Que faut-il documenter ?

Dans un projet Java, on documente en priorité :

— les classes ;
— les constructeurs ;
— les méthodes publiques ;
— les interfaces ;
— les méthodes dont le rôle n'est pas immédiatement évident.

En revanche, il n'est pas toujours nécessaire de documenter chaque ligne de code ou chaque accesseur trivial si leur rôle est évident.

Ce qu'il faut écrire dans une JavaDoc

Une bonne JavaDoc doit expliquer :

— le rôle de l'élément documenté ;
— les paramètres attendus ;
— le résultat produit ;
— les conditions particulières ;
— les erreurs possibles.

Il est préférable d'expliquer l'intention et l'utilisation d'une méthode, plutôt que de répéter mot à mot ce que le code montre déjà.

Par exemple, cette JavaDoc est peu utile :

```
/**
 * Retourne le nom.
 */
public String getNom() { ... }
```

Elle peut être acceptable pour un cours d'introduction, mais dans un projet réel on préférera une documentation plus informative lorsque c'est nécessaire.

Générer une documentation HTML

JavaDoc permet de produire automatiquement une documentation au format HTML.

En ligne de commande, il est possible d'utiliser l'outil `javadoc` :

```
javadoc *.java
```

Ou, dans un projet organisé en packages :

```
javadoc -d doc nomdupackage/*.java
```

Cette commande génère un ensemble de pages HTML documentant les classes, les méthodes et les attributs publics.

Le dossier `doc` contient alors les fichiers de documentation qu'il suffit d'ouvrir dans un navigateur.

Documentation HTML et PDF

La génération standard de JavaDoc produit une documentation **HTML**. C'est le format le plus courant, car il permet :

— une navigation simple entre classes et méthodes ;
— des liens cliquables ;
— une consultation facile dans un navigateur.

Si l'on souhaite obtenir une documentation au format PDF, il faut généralement passer par une étape supplémentaire, par exemple :

— générer d'abord la documentation HTML ;
— puis l'imprimer ou la convertir en PDF avec un navigateur ou un outil adapté.

En pratique, dans les projets Java, la documentation JavaDoc est presque toujours utilisée en HTML.

3.8.3 Aide contextuelle dans les éditeurs

L'un des grands avantages de JavaDoc est son intégration dans les environnements de développement comme **Eclipse** ou **IntelliJ IDEA**.

Lorsqu'une méthode est documentée, l'IDE peut afficher automatiquement cette documentation :

— au survol de la méthode ;
— pendant l'autocomplétion ;
— dans les panneaux d'aide contextuelle.

Cela améliore fortement le confort de développement. Le programmeur peut comprendre l'utilisation d'une méthode sans avoir à ouvrir sa définition.

JavaDoc joue donc un double rôle :

— fournir une documentation globale du projet ;
— servir d'aide immédiate pendant l'écriture du code.

3.8.4 Exemple d'intérêt dans un IDE

Si une méthode est documentée ainsi :

```
/**
 * Ajoute un étudiant à la liste.
 *
 * @param etudiant l'étudiant à ajouter
 * @throws IllegalArgumentException si l'étudiant vaut null
 */
```

```
public void ajouter(Etudiant etudiant) {
    ...
}
```

alors, dans un IDE, un développeur verra apparaître cette description lorsqu'il utilisera la méthode `ajouter()`.

Cela permet de comprendre immédiatement :

— le rôle de la méthode ;
— le paramètre attendu ;
— l'erreur possible.

3.8.5 Bonnes pratiques

Quelques recommandations simples :

— documenter les classes et les méthodes importantes ;
— utiliser les balises `@param`, `@return` et `@throws` lorsque c'est pertinent ;
— rédiger des phrases courtes et précises ;
— expliquer l'utilité d'une méthode, pas seulement répéter son nom ;
— maintenir la JavaDoc à jour lorsque le code évolue.

3.8.6 À retenir

JavaDoc est un outil central de la qualité du code Java. Il permet de documenter les classes et les méthodes directement dans le code source, de générer automatiquement une documentation HTML et d'améliorer l'aide contextuelle dans les éditeurs de développement.

Dans un projet sérieux, JavaDoc fait partie des bonnes pratiques de rigueur et de maintenance.

3.9 Contrôle de version

Dans un projet professionnel, le code est toujours géré avec un système de contrôle de version.

Le plus utilisé aujourd'hui est **Git**.

Un système de version permet notamment :

— de conserver l'historique des modifications
— de travailler à plusieurs
— de revenir à une version précédente en cas de problème

3.10 Exercices d'application

Exercice 3.1 : Correction d'une classe mal conçue

On vous fournit la classe suivante :

```java
public class Livre {

    public String titre;
    public String auteur;
    public boolean emprunte;

    public Livre(String t, String a) {
        titre = t;
        auteur = a;
        emprunte = false;
    }

    public void emprunter() {
        emprunte = true;
        System.out.println("Livre emprunte");
    }

}
```

Ce code ne respecte pas plusieurs bonnes pratiques vues dans ce chapitre.

Travail demandé

Corriger la classe afin de respecter les règles suivantes :

— les attributs doivent être `private` ;
— créer des setters pour modifier les attributs ;
— le constructeur doit passer par les setters ;
— supprimer tout affichage dans la classe métier ;
— utiliser des noms de paramètres explicites.

Extension : Ajouter une méthode `rendre()` permettant de rendre le livre.

Exercice 3.2 : Correction d'un programme mal structuré

On vous fournit le programme suivant :

```java
public class Etudiant {

    public String nom;
    public double moyenne;

    public Etudiant(String n, double m) {
        nom = n;
        moyenne = m;
    }

    public void afficher() {
        System.out.println(nom + " : " + moyenne);
    }

    public void ajouterPoints(double p) {
        moyenne = moyenne + p;
    }

}
```

Ce code ne respecte pas plusieurs bonnes pratiques vues dans ce chapitre.

Travail demandé :

Corriger la classe afin de respecter les règles suivantes :

- les attributs doivent être `private` ;
- créer des setters pour modifier les attributs ;
- le constructeur doit passer par les setters ;
- supprimer tout affichage dans la classe métier ;
- utiliser des noms de paramètres explicites.
- Ajouter une méthode permettant de vérifier si l'étudiant a la moyenne (note supérieure ou égale à 10).

Extension (pratique professionnelle) :

Une fois la correction terminée :

- créer un dépôt Git pour votre projet ;
- ajouter les fichiers du projet dans le dépôt ;
- effectuer un premier commit avec un message clair (par exemple : `Correction de la classe Etudiant`) ;
- publier le projet sur votre compte **GitHub**.

Cette étape permet de sauvegarder votre travail et de suivre l'évolution du code, comme cela se fait dans un projet professionnel.

Chapitre 4

Structures de contrôle

4.1 Les structures conditionnelles

Les structures conditionnelles permettent d'exécuter des blocs d'instructions en fonction d'une condition booléenne.

4.1.1 Structure if...else

La forme la plus simple est la structure à deux conditions :

```
if (condition) {
    instructions si condition vraie;
} else {
    instructions si condition fausse;
}
```

Exemple :

```
int j = 20;

if (j % 2 == 0) {
    System.out.println("Nombre pair");
} else {
    System.out.println("Nombre impair");
}
```

Structure à conditions multiples

Lorsque plusieurs cas sont possibles, on utilise `else if` :

```
int j = 15;

if (j > 0) {
```

```
    System.out.println("Nombre positif");
} else if (j < 0) {
    System.out.println("Nombre négatif");
} else {
    System.out.println("Zéro");
}
```

Conditions imbriquées

Une structure conditionnelle peut contenir d'autres conditions.

```
int age = 20;
int poids = 80;

if (age >= 18) {
    if (poids > 50) {
        System.out.println("Don autorisé");
    } else {
        System.out.println("Poids insuffisant");
    }
} else {
    System.out.println("Mineur");
}
```

L'imbrication doit rester lisible. Une profondeur excessive nuit à la maintenabilité.

4.1.2 La structure switch

La structure `switch` permet de choisir une action à exécuter en fonction de la valeur d'une variable. Elle constitue souvent une alternative plus lisible à une succession de `if` / `else if`.

Elle est particulièrement adaptée lorsque l'on doit tester plusieurs valeurs possibles d'une même variable.

```
int jour = 2;

switch (jour) {

    case 1:
        System.out.println("Lundi");
        break;

    case 2:
```

```
10          System.out.println("Mardi");
11          break;
12
13      default:
14          System.out.println("Jour inconnu");
15  }
```

Dans cet exemple, la variable `jour` vaut 2. Le programme affichera donc : **Mardi**

Le rôle de `break`

L'instruction `break` permet de sortir immédiatement du `switch`.

Si `break` est oublié, les instructions des cas suivants seront également exécutées. Ce comportement est appelé *fall-through*.

```
1  int jour = 1;
2
3  switch (jour) {
4
5      case 1:
6          System.out.println("Lundi");
7
8      case 2:
9          System.out.println("Mardi");
10
11 }
```

Dans cet exemple, le programme affichera :

```
Lundi
Mardi
```

En effet, sans `break`, l'exécution continue dans les cas suivants.

Il est donc généralement recommandé d'utiliser `break` à la fin de chaque cas afin d'éviter l'exécution involontaire des instructions suivantes.

Cependant, dans certains cas particuliers, on peut volontairement omettre `break` afin de faire exécuter plusieurs cas successivement.

Par exemple :

```
1  int note = 18;
2
3  switch (note) {
4
5      case 20:
```

```
    case 19:
    case 18:
        System.out.println("Excellent");
        break;

    case 17:
    case 16:
    case 15:
        System.out.println("Très bien");
        break;
}
```

Dans cet exemple, plusieurs valeurs partagent le même traitement. Le fait de ne pas utiliser `break` entre ces cas permet de regrouper plusieurs conditions.

4.2 Les structures itératives

Les boucles permettent de répéter un bloc d'instructions.

4.2.1 La boucle while

La boucle `while` vérifie la condition avant d'exécuter le bloc.

```
int i = 0;

while (i <= 10) {
    System.out.println("Valeur : " + i);
    i++;
}
```

Si la condition est fausse dès le départ, le bloc ne s'exécute jamais.

4.2.2 La boucle do...while

Contrairement à `while`, la boucle `do...while` s'exécute au moins une fois.

```
int i = 0;

do {
    System.out.println("Valeur : " + i);
    i++;
} while (i <= 10);
```

La condition est testée après la première exécution.

4.2.3 La boucle for

La boucle `for` regroupe initialisation, condition et incrémentation.

```
for (int i = 1; i <= 10; i++) {
    System.out.println("Ligne " + i);
}
```

Syntaxe générale :

```
for (initialisation; condition; incrementation) {
    instructions;
}
```

Il est possible d'utiliser plusieurs variables :

```
for (int i = 0, j = 2; (i < 10 && j < 6); i++, j += 2) {
    System.out.println("i=" + i + ", j=" + j);
}
```

4.2.4 Boucles sur séquences prédéfinies

Lorsque la séquence est connue (tableau), on peut utiliser la forme simplifiée :

```
int[] sequence = {8, 2, 9, 6, 12, 4};

for (int v : sequence) {
    System.out.println("Valeur : " + v);
}
```

4.3 Instructions break et continue

break

L'instruction `break` permet d'interrompre immédiatement une boucle.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
}
```

continue

L'instruction `continue` permet de passer à l'itération suivante.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        continue;
    }
    System.out.println(i);
}
```

4.4 Attention aux boucles infinies

Une boucle infinie apparaît lorsque la condition d'arrêt n'est jamais atteinte. Le programme reste alors bloqué dans la boucle.

```
int i = 0;

while (i <= 10) {
    System.out.println(i);
}
```

Dans cet exemple, la variable `i` n'est jamais modifiée dans la boucle. La condition `i <= 10` reste donc toujours vraie, ce qui provoque une boucle infinie.

Notion de convergence

Pour qu'une boucle se termine, il faut que les variables utilisées dans la condition évoluent progressivement vers la condition d'arrêt.

On dit alors que la boucle **converge** vers sa condition de sortie.

Dans l'exemple précédent, il faudrait modifier la variable `i` à chaque itération :

```
int i = 0;

while (i <= 10) {
    System.out.println(i);
    i++;
}
```

À chaque tour de boucle, la variable `i` augmente. La condition finira donc par devenir fausse et la boucle s'arrêtera.

À retenir

Pour éviter les boucles infinies :

— la condition de la boucle doit pouvoir devenir fausse ;
— les variables utilisées dans la condition doivent être modifiées dans la boucle ;
— la boucle doit **converger** vers sa condition d'arrêt.

4.5 Exercices d'application

Exercice 4.1 : Analyse et validation d'un nombre

Écrire un programme qui :
— demande à l'utilisateur de saisir un nombre entier ;
— indique si le nombre est positif, négatif ou nul ;
— indique si le nombre est pair ou impair.
Contraintes :
— utiliser une structure `if...else if...else` ;
— ne pas dupliquer les conditions ;
— le code doit être correctement indenté.

Extension : Afficher un message spécifique si le nombre est à la fois positif et pair.

Exercice 4.2 : Table de multiplication contrôlée

Écrire un programme qui :
— demande un entier compris entre 1 et 10 ;
— vérifie la validité de la saisie ;
— affiche la table de multiplication correspondante ;
— propose à l'utilisateur de recommencer.
Contraintes :
— utiliser une boucle `do...while` pour garantir au moins une exécution ;
— utiliser une boucle `for` pour afficher la table ;
— contrôler la saisie avec une structure conditionnelle ;
— éviter toute boucle infinie.

Exemple attendu pour l'entrée 3 :

```
3 x 1 = 3
3 x 2 = 6
...
3 x 10 = 30
```

Exercice 4.3 : Parcours et filtrage

On dispose du tableau suivant :

```java
int[] notes = {12, 8, 15, 3, 19, 7, 14, 5};
```

Écrire un programme qui :

— parcourt le tableau ;
— affiche uniquement les notes supérieures ou égales à 10 ;
— ignore les notes strictement inférieures à 5 ;
— s'arrête immédiatement si une note égale à 19 est rencontrée.

Contraintes :

— utiliser une boucle `for` améliorée ;
— utiliser l'instruction `continue` ;
— utiliser l'instruction `break`.

Question d'analyse : Quel sera l'ordre exact des valeurs affichées ?

Chapitre 5

Méthodes et structuration du code

5.1 Méthodes de classe et programmation procédurale

Les méthodes utilisées dans ce chapitre sont déclarées avec le mot-clé `static`.

Il s'agit de méthodes de classe, c'est-à-dire qu'elles appartiennent à la classe elle-même et non à un objet particulier.

Elles peuvent être appelées directement depuis la méthode `main` sans créer d'instance de la classe.

```
public static int addition(int a, int b) {
    return a + b;
}
```

Dans ce contexte, ces méthodes sont utilisées de manière similaire à des fonctions dans un *langage procédural*.

Cette approche correspond à la **programmation procédurale** :

— le programme est structuré en fonctions ;

— les données sont manipulées par ces fonctions ;

— l'exécution suit une logique séquentielle.

Il s'agit d'une étape importante pour apprendre à structurer un programme.

La programmation orientée objet adopte une logique différente :

— les données et les comportements sont regroupés dans des objets ;

— les méthodes ne sont plus appelées de manière globale ;

— le comportement dépend de l'état d'un objet.

La différence fondamentale est la suivante :

En programmation procédurale, on applique des fonctions à des données.

En programmation orientée objet, ce sont les objets qui portent leurs propres comportements.

Cette évolution de paradigme sera étudiée dans le chapitre suivant.

5.2 Pourquoi utiliser des méthodes ?

Un programme écrit uniquement dans la méthode `main` devient rapidement :

— difficile à lire ;
— difficile à tester ;
— difficile à maintenir ;
— difficile à réutiliser.

Une méthode permet de regrouper un ensemble d'instructions réalisant une tâche précise.

Une méthode = une responsabilité.

5.3 Déclaration d'une méthode

Syntaxe générale :

```
modificateur typeRetour nomMethode(parametres) {
    instructions;
    return valeur;
}
```

Exemple simple :

```
public static int addition(int a, int b) {
    return a + b;
}
```

Appel de la méthode :

```
int resultat = addition(5, 3);
System.out.println(resultat);
```

5.4 Paramètres et valeur de retour

Paramètres

Les paramètres permettent de transmettre des informations à la méthode.

```
public static void afficherMessage(String message) {
    System.out.println(message);
}
```

Valeur de retour

Une méthode peut retourner une valeur au programme appelant.

Le type de cette valeur doit obligatoirement être indiqué dans la déclaration de la méthode.

```
public static int addition(int a, int b) {
    return a + b;
}
```

Ici, le type de retour est `int`. La méthode doit donc obligatoirement retourner une valeur de type `int`.

Règle fondamentale :

Le type indiqué avant le nom de la méthode correspond à la nature de la valeur retournée.

Exemples de types de retour possibles :

- `int` : nombre entier ;
- `double` : nombre décimal ;
- `boolean` : vrai ou faux ;
- `String` : chaîne de caractères ;
- un objet (par exemple `Livre`) ;
- `void` : aucune valeur retournée.

Exemple avec un booléen :

```
public static boolean estMajeur(int age) {
    return age >= 18;
}
```

Ici, la méthode retourne uniquement deux valeurs possibles : `true` ou `false`.

Une méthode qui déclare un type de retour différent de `void` doit obligatoirement contenir une instruction `return` retournant une valeur compatible avec le type déclaré.

```
public static String saluer(String nom) {
    return "Bonjour " + nom;
}
```

Si aucune valeur n'est retournée alors que le type n'est pas `void`, le compilateur génère une erreur.

La maîtrise des valeurs de retour est essentielle pour permettre :

- la réutilisation du code ;
- la composition de traitements ;
- la testabilité des méthodes.

5.5 Effets de bord et fonctions pures

Qu'est-ce qu'un effet de bord ?

Une méthode produit un **effet de bord** lorsqu'elle modifie un élément extérieur à son propre calcul.

Exemples d'effets de bord :

— affichage dans la console ;
— modification d'une variable globale ;
— écriture dans un fichier ;
— modification d'un objet reçu en paramètre.

Exemple :

```
private static int carre(int nombre) {
    int resultat = nombre * nombre;
    System.out.println("Résultat : " + resultat);
    return resultat;
}
```

Cette méthode :

— calcule ;
— affiche ;
— retourne.

Elle possède donc un effet de bord (l'affichage).

Pourquoi éviter les effets de bord ?

Les effets de bord rendent le code :

— plus difficile à tester ;
— moins prévisible ;
— plus difficile à réutiliser ;
— plus difficile à maintenir.

Prenons un exemple concret.

Si une méthode calcule un résultat et l'affiche directement dans la console, elle devient dépendante de ce mode d'affichage.

Or, demain, le besoin peut évoluer :

— afficher le résultat dans une interface graphique (JavaFX) ;
— envoyer le résultat à une autre méthode ;
— enregistrer le résultat dans une base de données ;
— écrire le résultat dans un fichier ;
— exposer le résultat via une API.

Si la méthode contient un `System.out.println`, il faudra la modifier pour chaque nouveau contexte.

En revanche, si la méthode se contente de retourner une valeur :

```
private static int carre(int nombre) {
    return nombre * nombre;
}
```

Alors le résultat peut être utilisé librement :

— affiché en console ;
— affiché dans une interface graphique ;
— stocké en base de données ;
— transmis à une autre méthode ;
— testé automatiquement.

La méthode devient indépendante du contexte d'utilisation.

Principe professionnel

Une méthode doit produire une valeur, pas décider de la manière dont elle sera utilisée.

Cette séparation rend le code plus flexible, plus évolutif et plus professionnel.

Il est recommandé de :

— séparer le calcul de l'affichage ;
— limiter les effets de bord ;
— rendre les méthodes aussi pures que possible.

Ce principe sera fondamental pour les tests unitaires et la programmation orientée objet étudiés dans les chapitres suivants.

5.6 Visibilité des méthodes : public et private

En Java, une méthode possède un niveau de visibilité qui détermine où elle peut être utilisée.

— **public** : la méthode est accessible depuis n'importe quelle autre classe ;
— **private** : la méthode est accessible uniquement à l'intérieur de la classe.

Exemple :

```
public class Calculatrice {

    public static void main(String[] args) {
        int resultat = addition(5, 3);
        System.out.println(resultat);
    }

```

```
    private static int addition(int a, int b) {
        return a + b;
    }
}
```

Dans cet exemple, la méthode `addition` est déclarée `private` car elle est utilisée uniquement dans la classe `Calculatrice`.

Principe professionnel :

Une méthode ne doit pas être plus visible que nécessaire.

Rendre une méthode `public` sans raison augmente le couplage et diminue la sécurité du code.

Même dans une approche procédurale utilisant des méthodes `static`, il est important de respecter ce principe d'encapsulation.

La gestion fine de la visibilité deviendra essentielle en programmation orientée objet.

5.7 Validation des paramètres d'une méthode

Une méthode professionnelle ne doit jamais supposer que les données reçues sont valides.

Prenons l'exemple suivant :

```
if (tableau == null || tableau.length == 0) {
    throw new IllegalArgumentException(
        "Le tableau ne peut pas être vide."
    );
}
```

Que fait exactement ce code ?

— Le mot-clé `throw` signifie ń lancer ż.
— Il interrompt immédiatement l'exécution normale de la méthode.
— Il crée un objet représentant une erreur.

`IllegalArgumentException` est une classe prédéfinie en Java.

Elle signifie que : « Un argument fourni à la méthode est invalide. »

La chaîne de caractères : « Le tableau ne peut pas être vide. » est le message derreur qui sera affiché si le programme sarrête à cet endroit.

Que se passe-t-il ensuite ?

Lorsque l'instruction `throw` est exécutée :

— la méthode s'arrête immédiatement ;
— le programme remonte l'information d'erreur ;
— si l'erreur n'est pas interceptée, le programme se termine.

Pourquoi est-ce une bonne pratique ?

Parce quune méthode bien conçue :

— vérifie ses paramètres ;
— refuse de travailler avec des données incohérentes ;
— signale clairement le problème.

Cela évite des erreurs plus difficiles à comprendre plus tard (comme une division par zéro ou un `NullPointerException`).

Nous étudierons en détail le mécanisme des exceptions dans le chapitre consacré à la robustesse des applications.

5.8 Factorisation d'un programme

Exemple sans structuration :

```
int age = 20;

if (age >= 18) {
    System.out.println("Majeur");
}
```

Version structurée :

```
public static boolean estMajeur(int age) {
    return age >= 18;
}

public static void main(String[] args) {
    int age = 20;

    if (estMajeur(age)) {
        System.out.println("Majeur");
    }
}
```

La logique devient réutilisable et testable.

5.9 Organisation logique d'un programme

Un programme structuré respecte les principes suivants :

— une méthode réalise une tâche précise ;
— une méthode ne doit pas dépasser une taille raisonnable ;
— les noms doivent exprimer l'intention ;
— la duplication de code doit être évitée.

5.10 Exemple complet structuré

```java
import java.util.Scanner;

public class CalculatriceSimple {

    public static int addition(int a, int b) {
        return a + b;
    }

    public static int soustraction(int a, int b) {
        return a - b;
    }

    public static void afficherResultat(int resultat) {
        System.out.println("Résultat : " + resultat);
    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Premier nombre : ");
        int a = scanner.nextInt();
        System.out.print("Second nombre : ");
        int b = scanner.nextInt();
        int resultat = addition(a, b);
        afficherResultat(resultat);

        scanner.close();
    }
}
```

On observe :

— séparation des responsabilités ;

— réutilisabilité ;

— lisibilité améliorée.

5.11 Exercices d'application

Exercice 5.1 : Première méthode

Écrire une méthode :

— nommée `carre` ;

— recevant un entier en paramètre ;

— retournant le carré de cet entier.

Dans la méthode `main`, appeler cette méthode avec plusieurs valeurs et afficher les résultats.

Contraintes :

— la méthode doit être `private static` ;

— elle doit retourner un `int` ;

— aucun affichage ne doit être réalisé dans la méthode.

Exercice 5.2 : Méthode de calcul avec retour

Écrire une méthode :

— nommée `calculerMoyenne` ;

— recevant un tableau d'entiers ;

— retournant un `double`.

La méthode doit :

— calculer la somme des éléments ;

— retourner la moyenne.

Dans `main` :

— créer un tableau de notes ;

— afficher la moyenne retournée.

Chapitre 6

Modélisation orientée objet

6.1 Du procédural à l'orienté objet

Jusqu'à présent, nous avons utilisé des méthodes `static`. Ce mode de programmation correspond à une approche dite **procédurale**.

Contexte historique

Dans les années **1960-1970**, les programmes informatiques deviennent de plus en plus complexes.

Les premiers langages (*Fortran, C, Pascal*) sont principalement procéduraux : on écrit une suite d'instructions et de fonctions qui manipulent des données globales.

Ce modèle fonctionne pour de petits programmes. Mais lorsque les applications deviennent volumineuses :

— le code devient difficile à maintenir ;
— les dépendances entre fonctions se multiplient ;
— les données sont accessibles partout ;
— les modifications provoquent des effets de bord imprévus.

À partir des années 1980, un autre phénomène apparaît : le coût du développement et surtout de la maintenance logicielle dépasse progressivement le coût du matériel.

Autrement dit, le problème n'est plus uniquement technique, il devient **économique**.

Une application doit pouvoir :

— évoluer dans le temps ;
— être maintenue par différentes équipes ;
— être corrigée sans tout réécrire ;
— être comprise plusieurs années après sa création.

Les ingénieurs se retrouvent face à un problème majeur : Comment maîtriser la complexité croissante des logiciels tout en réduisant les coûts de maintenance ?

C'est dans ce contexte que naît la Programmation Orientée Objet, notamment avec le langage *Simula* (1967), puis *Smalltalk* dans les années 1970.

Dans les années 1980, *C++* popularise la POO dans le monde industriel en l'intégrant au langage C.

Enfin, **Java**, lancé officiellement en 1995 par *Sun Microsystems*, va démocratiser la Programmation Orientée Objet dans le développement d'applications professionnelles et dans le monde de l'entreprise.

Son objectif était clair : « Write Once, Run Anywhere »

Java apporte alors :

— une machine virtuelle (JVM) et donc la portabilité ;
— une gestion automatique de la mémoire contrairement au C ;
— une bibliothèque standard riche ;
— une approche fortement orientée objet.

La POO propose une réponse méthodologique : structurer les programmes autour d'entités autonomes, regroupant données et comportements, afin de mieux contrôler la complexité.

6.2 Définir une classe

Une **classe** est une structure fondamentale de la Programmation Orientée Objet. Elle définit un **nouveau type** dans le programme.

Autrement dit, une classe permet de créer des objets ayant une même structure et un même comportement.

Une classe décrit :

— les **données** qu'un objet possédera (les **attributs**) ;
— les **actions** qu'il pourra réaliser (les **méthodes**).

Classe et objet : ne pas confondre

Il est essentiel de distinguer :

— la **classe** : le modèle, le plan de construction ;
— l'**objet** : l'instance concrète créée à partir de ce modèle.

On peut comparer une classe au plan d'une maison, et un objet à une maison effectivement construite.

Important : la classe ne représente pas un élément unique, mais une catégorie d'éléments partageant les mêmes caractéristiques.

Lien avec la modélisation métier

En développement professionnel, une classe représente généralement une **entité métier**.

Par exemple :

— un `Etudiant` ;

— un `Client` ;

— une `Commande` ;

— un `Produit`.

La classe permet donc de modéliser le monde réel dans le programme informatique.

Exemple simple

Modélisons un étudiant.

```
public class Etudiant {

    private String nom;
    private int age;

}
```

Cette classe définit :

— deux attributs : `nom` et `age` ;

— un nouveau type appelé `Etudiant`.

On pourra ensuite créer autant d'objets `Etudiant` que nécessaire.

6.3 Le mot-clé `private` et le principe d'encapsulation

Les modificateurs de visibilité

En Java, chaque attribut ou méthode peut être précédé d'un **modificateur de visibilité**.

Les principaux sont :

— `public` ;

— `private` ;

— `protected` (que nous verrons plus tard).

Ces mots-clés déterminent **qui a le droit d'accéder** à un élément.

Que signifie `private` ?

Lorsqu'un attribut est déclaré `private`, il n'est accessible **que depuis la classe elle-même**.

Exemple :

```
public class Etudiant {

    private int age;

}
```

Dans ce cas, il est impossible d'écrire :

```
etudiant.age = 25; // interdit
```

L'attribut est protégé contre toute modification directe extérieure.

Pourquoi protéger les attributs ?

Si les attributs étaient publics, n'importe quelle partie du programme pourrait modifier l'objet, sans contrôle.

Exemple problématique :

```
etudiant.age = -10;
```

On obtient alors un objet dans un état incohérent.

Un objet doit rester cohérent tout au long de sa vie.

Le principe d'encapsulation

L'**encapsulation** consiste à :

— regrouper les données et les comportements dans une même classe ;
— protéger les données contre les accès directs ;
— contrôler les modifications via des méthodes.

On peut résumer ainsi :

Les données sont cachées, les comportements sont exposés.

6.4 Instanciation d'un objet

Créer un objet s'appelle **instancier** une classe.

```
Etudiant etudiant = new Etudiant();
```

Le mot-clé `new` :

— alloue de la mémoire ;
— crée l'objet ;
— appelle le constructeur.

La variable contient une **référence** vers l'objet.

6.5 Getters et setters

Les **getters** permettent de lire un attribut. Les **setters** permettent de modifier un attribut en appliquant des règles.

```java
public class Etudiant {

    private String nom;
    private int age;

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        if (nom == null || nom.isBlank()) {
            return;
        }
        this.nom = nom;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age < 0) {
            return;
        }
        this.age = age;
    }
}
```

Point essentiel : la validation est centralisée dans les setters.

6.6 Le constructeur

Le **constructeur** initialise l'objet lors de sa création.

```java
Etudiant e1 = new Etudiant("Alice", 20);
```

Bonne pratique : le constructeur doit passer par les setters afin de conserver les règles de validation.

```java
public class Etudiant {

    private String nom;
    private int age;

    public Etudiant(String nom, int age) {
        setNom(nom);
        setAge(age);
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        if (nom == null || nom.isBlank()) {
            return;
        }
        this.nom = nom;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age < 0) {
            return;
        }
        this.age = age;
    }
}
```

Pourquoi ? Si le constructeur affecte directement les attributs :

```java
this.age = age;
```

il contourne les règles de validation.

En appelant les setters :

— les contrôles sont appliqués systématiquement ;

— l'objet ne peut pas être créé dans un état invalide ;

— le comportement reste cohérent.

6.7 La référence `this`

Que représente `this` ?

En Java, le mot-clé `this` désigne **l'objet courant**.

Autrement dit, `this` fait référence à l'instance à l'intérieur de laquelle le code est en train de s'exécuter.

Chaque fois qu'un objet appelle une méthode, Java fournit implicitement une référence vers cet objet. Cette référence s'appelle `this`.

6.7.1 Pourquoi a-t-on besoin de `this` ?

Considérons l'exemple suivant :

```
public class Etudiant {

    private String nom;

    public void setNom(String nom) {
        nom = nom;
    }
}
```

Ce code semble correct, mais il ne fonctionne pas comme prévu.

Dans cette méthode :

— le paramètre `nom` est une variable locale ;

— l'attribut `nom` de la classe porte le même nom.

L'instruction :

```
nom = nom;
```

affecte simplement la variable locale à elle-même. L'attribut de l'objet n'est jamais modifié.

La solution : utiliser `this`

Pour accéder à l'attribut de l'objet, on utilise `this`.

```
public void setNom(String nom) {
    this.nom = nom;
}
```

Ici :

— `this.nom` désigne l'attribut de l'objet ;

— `nom` désigne le paramètre de la méthode.

Lecture correcte : « L'attribut `nom` de l'objet courant reçoit la valeur du paramètre `nom`. »

Quand utilise-t-on `this` ?

`this` est utilisé principalement dans deux situations :

— lorsqu'un paramètre porte le même nom qu'un attribut ;

— pour clarifier explicitement que l'on manipule l'objet courant.

Conseil de bonne pratique :

Même lorsque ce n'est pas strictement obligatoire, il est recommandé d'utiliser `this` à chaque fois que l'on manipule un attribut de la classe.

Cela permet :

— d'améliorer la lisibilité du code ;

— d'éviter toute ambiguïté ;

— de rendre explicite la distinction entre attribut et variable locale.

Exemple complet

```java
public class Etudiant {

    private String nom;
    private int age;

    public Etudiant(String nom, int age) {
        setNom(nom);
        setAge(age);
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        if (nom == null || nom.isBlank()) {
            return;
        }
        this.nom = nom;
    }

    public int getAge() {
```

```
23          return age;
24      }
25
26      public void setAge(int age) {
27          if (age < 0) {
28              return;
29          }
30          this.age = age;
31      }
32  }
```

On peut résumer ainsi :

`this` = l'objet courant.

6.8 Exercices d'application

Exercice 6.1 : Modélisation simple

Créer une classe `Livre` représentant un livre.

La classe doit contenir :

— un attribut `titre` de type `String` ;

— un attribut `auteur` de type `String` ;

— un attribut `nombrePages` de type `int`.

Contraintes :

— tous les attributs doivent être `private` ;

— créer les **getters** et **setters** correspondants ;

— utiliser `this` dans les setters.

Dans une classe `Main` :

— créer un objet `Livre` ;

— initialiser ses valeurs via les setters ;

— afficher le titre du livre.

Exercice 6.2 : Encapsulation et validation

Créer une classe `CompteBancaire`.

La classe doit contenir :

— un attribut `titulaire` de type `String` ;

— un attribut `solde` de type `double`.

Contraintes :

— les attributs doivent être `private` ;

— le `solde` ne peut pas être négatif ;

— la validation doit être réalisée dans le setter ;

— utiliser systématiquement `this`.

Ajouter :

— un constructeur recevant `titulaire` et `solde` ;

— le constructeur doit passer par les setters.

Tester dans une classe `Main`.

Exercice 6.3 : Modélisation métier complète

Créer une classe `Etudiant` représentant un étudiant.

La classe doit contenir :

— `nom` (`String`) ;

— `prenom` (`String`) ;
— `age` (`int`) ;
— `moyenne` (`double`).

Contraintes :

— tous les attributs doivent être `private` ;
— créer les getters et setters ;
— l'âge ne peut pas être négatif ;
— la moyenne doit être comprise entre 0 et 20 ;
— le constructeur doit utiliser les setters.

Ajouter une méthode :

— `afficherInformations()` affichant les données de l'étudiant.

Affichage d'un objet et introduction à `toString()`

Dans l'exercice précédent, on demande d'implémenter une méthode :

— `afficherInformations()` affichant les données de l'étudiant.

Par exemple :

```
public void afficherInformations() {
    System.out.println("Nom : " + nom);
    System.out.println("Age : " + age);
}
```

Cette approche fonctionne, mais en Programmation Orientée Objet, il existe une méthode plus adaptée : `toString()`.

Qu'est-ce que `toString()` ?

Toutes les classes Java héritent implicitement de la classe `Object`. Cette classe possède une méthode :

```
public String toString()
```

Son rôle est de fournir une représentation textuelle de l'objet.

On peut donc redéfinir cette méthode dans notre classe :

```
@Override
public String toString() {
    return "Etudiant[nom=" + nom + ", age=" + age + "]";
}
```

Ensuite, il suffit d'écrire :

```
System.out.println(etudiant);
```

Java appellera automatiquement la méthode `toString()`.

Remarque importante :

Pour l'instant, la méthode `afficherInformations()` permet de comprendre la structure d'une classe.

Nous reviendrons plus en détail sur `toString()` lorsque nous étudierons l'héritage et la classe `Object`.

Chapitre 7

Architecture applicative et séparation des couches

7.1 Pourquoi parler d'architecture ?

Jusqu'à présent, nous avons appris à écrire des classes. Mais une application professionnelle ne se résume pas à une seule classe.

Lorsqu'un projet devient plus important, une question essentielle apparaît :

Comment organiser le code pour qu'il reste lisible, maintenable et évolutif ?

La réponse passe par la notion d'**architecture**.

Une architecture logicielle définit la manière dont les différentes parties d'une application sont organisées et communiquent entre elles.

7.2 Le problème d'un code mal structuré

Considérons un programme qui mélange :

— la saisie utilisateur ;
— la logique métier ;
— l'affichage ;
— l'accès aux fichiers.

Dans ce cas :

— toute modification impacte plusieurs endroits ;
— le code devient difficile à tester ;
— l'application devient fragile.

Mélanger les responsabilités est l'une des principales causes de complexité.

7.3 Principe fondamental : la séparation des responsabilités

Une application bien conçue sépare clairement :
— la **représentation des données** ;
— la **logique métier** ;
— l'**interface utilisateur**.
Ce principe est fondamental en ingénierie logicielle.

Il prépare naturellement à un modèle d'architecture très répandu : le **MVC**.

7.4 Le modèle MVC

MVC signifie :
— **Model** ;
— **View** ;
— **Controller**.

Le Model

Le **Model** représente :
— les entités métier ;
— les règles de gestion ;
— l'état de l'application.
Exemple : `Etudiant`, `CompteBancaire`, `Produit`.

La View

La **View** correspond à l'interface utilisateur.
Elle peut être :
— une console ;
— une interface graphique ;
— une page web.
La View affiche les données mais ne contient pas la logique métier.

Le Controller

Le **Controller** joue le rôle d'intermédiaire.
Il :
— reçoit les actions de l'utilisateur ;
— appelle le Model ;
— met à jour la View.

7.5 Exemple cohérent d'architecture MVC

Imaginons une application simple de gestion d'étudiants.

Nous allons distinguer clairement les trois rôles :

— le **Model** ;
— le **Controller** ;
— la **View**.

1. Le Model

Le **Model** contient les données et la logique métier.

```
public class Etudiant {

    private String nom;
    private double moyenne;

    public Etudiant(String nom, double moyenne) {
        setNom(nom);
        setMoyenne(moyenne);
    }

    public String getNom() {
        return this.nom;
    }

    public void setNom(String nom) {
        if (nom != null && !nom.isBlank()) {
            this.nom = nom;
        }
    }

    public double getMoyenne() {
        return this.moyenne;
    }

    public void setMoyenne(double moyenne) {
        if (moyenne >= 0 && moyenne <= 20) {
            this.moyenne = moyenne;
        }
    }

```

```
    public boolean estAdmis() {
        return this.moyenne >= 10;
    }
}
```

Dans cet exemple :

— la classe contient les données ;

— la méthode `estAdmis()` représente une règle métier.

2. Le Controller

Le **Controller** orchestre les interactions entre le Model et la View.

```
public class EtudiantController {

    public String traiterAdmission(Etudiant etudiant) {

        if (etudiant.estAdmis()) {
            return etudiant.getNom() + " est admis.";
        } else {
            return etudiant.getNom() + " est ajourné.";
        }
    }
}
```

Le Controller :

— appelle les méthodes du Model ;

— prépare une réponse ;

— ne contient pas de logique d'affichage complexe.

3. La View

La **View** est responsable de l'affichage.

```
public class Main {

    public static void main(String[] args) {

        Etudiant etudiant = new Etudiant("Alice", 12.5);
        EtudiantController controller = new EtudiantController();
        String message = controller.traiterAdmission(etudiant);
        System.out.println(message);
    }
}
```

La View :
— instancie les objets ;
— appelle le Controller ;
— affiche le résultat.

Analyse

Dans cette organisation :
— le **Model** contient la logique métier ;
— le **Controller** coordonne ;
— la **View** affiche.

Aucune couche ne mélange les responsabilités.
C'est précisément cette séparation qui rend l'application maintenable et évolutive.

7.6 Pourquoi cette séparation est essentielle ?

La séparation des couches permet :
— de modifier l'interface sans toucher au métier ;
— de tester la logique métier indépendamment ;
— de faire évoluer l'application plus facilement ;
— de réduire les effets de bord.
Une application professionnelle est toujours organisée en couches.

7.7 Préparation à JavaFX

Dans les chapitres suivants, nous utiliserons **JavaFX** pour créer une interface graphique.
JavaFX s'appuie naturellement sur le modèle MVC.

Il est donc indispensable de comprendre dès maintenant :
— que la logique métier ne doit pas être placée dans la View ;
— que le Controller ne doit pas contenir toute la logique ;
— que le Model doit rester indépendant de l'interface graphique.

7.8 Exercices d'application

Exercice 7.1 : Refactorisation vers une architecture MVC

On vous fournit le programme suivant :

```java
public class GestionNote {

    public static void main(String[] args) {

        double moyenne = 12.5;

        if (moyenne >= 10) {
            System.out.println("Admis");
        } else {
            System.out.println("Ajourné");
        }
    }
}
```

Ce programme fonctionne correctement. Cependant, il mélange plusieurs responsabilités :

- la gestion des données ;
- la règle métier (admis / ajourné) ;
- l'affichage.

Travail demandé :

Restructurer ce programme afin de respecter une architecture **Model / Controller / View (MVC)**.

- Créer une classe `Etudiant` (Model) contenant l'attribut `moyenne` ;
- Ajouter une méthode `estAdmis()` dans le Model ;
- Créer une classe `EtudiantController` (Controller) ;
- Le Controller doit appeler le Model et retourner un message adapté ;
- Créer une classe `Main` (View) responsable uniquement de l'affichage.

Contraintes :

- Le Model ne doit contenir aucun `System.out.println` ;
- Le Controller ne doit pas effectuer d'affichage ;
- La View ne doit contenir aucune règle métier ;
- Les responsabilités doivent être clairement séparées.

Exercice 7.2 : Organisation en couches

On souhaite créer une petite application de gestion de produit.

Créer :

- une classe `Produit` (Model) contenant :
  - `nom` ;
  - `prix` ;
- une méthode `appliquerRemise(double pourcentage)` modifiant le prix ;

Puis :

- créer un `ProduitController` ;
- le Controller doit appeler la méthode du Model ;
- la classe `Main` doit uniquement :
  - créer le produit ;
  - appeler le Controller ;
  - afficher le résultat.

Contraintes :

- aucune logique métier ne doit se trouver dans `Main` ;
- aucune règle de calcul ne doit être écrite dans la View.

Exercice 7.3 : Mini-architecture complète

On souhaite modéliser une application simple de gestion de compte bancaire.

Créer :

- une classe `CompteBancaire` (Model) contenant :
  - `titulaire` ;
  - `solde` ;
  - une méthode `deposer(double montant)` ;
  - une méthode `retirer(double montant)` ;

Puis :

- créer une classe `CompteController` ;
- le Controller doit gérer les appels au Model ;
- la classe `Main` joue le rôle de View.

Contraintes :

- la View ne doit contenir aucun calcul ;
- le Controller ne doit pas afficher directement ;
- le Model ne doit pas contenir d'affichage console.

Chapitre 8

Héritage et polymorphisme

8.1 Pourquoi l'héritage ?

Lorsque plusieurs classes partagent des caractéristiques communes, la duplication de code devient dangereuse. Les principes de conception rappellent qu'il faut :

— rechercher un **couplage faible** ;
— rechercher une **cohésion forte** ;
— éviter la duplication ;
— faciliter les changements localisés.

L'héritage permet de factoriser un comportement commun.

L'héritage permet de spécialiser un comportement commun.

8.2 Relation est-un et a-un

L'héritage modélise une relation de spécialisation :

Une sous-classe est une forme spécialisée de la classe mère.

Exemple valide :

— Un `Chien` est un `Animal`.

Exemple invalide :

— Une `Voiture` n'est pas un `Moteur`.

Dans ce cas, il s'agit d'une relation **a-un** (composition).

Règle fondamentale : n'utiliser l'héritage que si la relation correspond réellement à « est-un ».

Remarque Si la relation entre deux classes n'est pas clairement une relation « est-un », l'héritage est probablement inadapté.

Dans ce cas, une autre approche de conception, appelée **composition**, sera plus pertinente.

Cette notion sera étudiée en détail au chapitre suivant, dans le cadre des principes SOLID et de la conception orientée objet avancée.

8.3 Définir une classe mère

```java
public class Animal {

    private String nom;

    public Animal(String nom) {
        setNom(nom);
    }

    public String getNom() {
        return this.nom;
    }

    public void setNom(String nom) {
        if (nom != null && !nom.isBlank()) {
            this.nom = nom;
        }
    }

    public void crier() {
        System.out.println("L'animal crie.");
    }
}
```

La classe `Animal` définit un comportement générique.

8.4 Créer une sous-classe

```java
public class Chien extends Animal {

    public Chien(String nom) {
        super(nom);
    }
```

```
6
7       @Override
8       public void crier() {
9           System.out.println("Le chien aboie.");
10      }
11  }
```

Points importants :

— `extends` indique l'héritage ;

— `super()` appelle le constructeur de la classe mère ;

— `@Override` sécurise la redéfinition.

L'annotation `@Override`

L'annotation `@Override` indique explicitement qu'une méthode redéfinit une méthode de la classe mère.

```
1  @Override
2  public void crier() {
3      System.out.println("Le chien aboie.");
4  }
```

Pourquoi est-ce important ? Sans `@Override`, le compilateur ne vérifie pas que la méthode existe réellement dans la classe mère.

Si l'on commet une erreur de signature :

```
1  public void crier(String message) {
2      System.out.println("Le chien aboie.");
3  }
```

Cette méthode ne redéfinit plus la méthode de la classe mère. Elle crée une **nouvelle méthode**.

Si la signature ne correspond pas exactement à celle de la classe mère, il ne s'agit plus d'une *redéfinition* (**override**), mais d'une *surcharge* (**overload**).

La méthode de la classe mère ne sera alors pas remplacée.

Il n'y aura donc pas de redéfinition effective, et le mécanisme de liaison dynamique ne s'appliquera pas à cette méthode. Si la signature ne correspond pas exactement à celle de la classe mère, il ne s'agit plus d'une redéfinition (override), mais d'une surcharge (overload). La méthode de la classe mère ne sera alors pas remplacée. Il n'y aura donc pas de redéfinition effective, et le mécanisme de liaison dynamique ne s'appliquera pas à cette méthode.

Avec `@Override`, le compilateur génère une erreur si la signature ne correspond pas exactement à celle de la classe mère.

`@Override` est une sécurité de compilation.

Règles de redéfinition

Pour qu'une méthode redéfinisse correctement une méthode héritée :
- le nom doit être identique ;
- les paramètres doivent être identiques ;
- le type de retour doit être compatible ;
- le niveau de visibilité ne peut pas être restreint ;
- la méthode ne doit pas être `final`.

Bonne pratique Dans un code professionnel, **toute méthode redéfinie doit être annotée avec `@Override`**.

8.5 Polymorphisme

Le polymorphisme signifie :

Un même message produit un comportement différent selon l'objet réel.

```
Animal a1 = new Chien("Rex");
Animal a2 = new Animal("Inconnu");

a1.crier();
a2.crier();
```

La méthode exécutée dépend du **type réel de l'objet**, et non du type de la variable. Il s'agit du mécanisme de **liaison dynamique**.

8.6 Classes abstraites

Une classe peut être déclarée abstraite :

```
public abstract class Animal {

    private String nom;

    public Animal(String nom) {
        setNom(nom); /
    }
```

```
    public String getNom() {
        return this.nom;
    }
    public void setNom(String nom) {
        if (nom != null && !nom.isBlank()) {
            this.nom = nom;
        }
    }
    public abstract void crier();
}
```

Une méthode abstraite :

— ne possède pas de corps ;
— doit être implémentée par les sous-classes ;
— force la spécialisation.

8.7 Principe de substitution de Liskov

Le principe de substitution de Liskov (LSP), formulé par *Barbara Liskov* en 1987, énonce que :

Un objet d'une sous-classe doit pouvoir remplacer un objet de la classe mère sans altérer le comportement attendu du programme.

Autrement dit, tout code utilisant la classe mère doit continuer à fonctionner correctement si l'on lui fournit une instance d'une sous-classe.

Interprétation concrète

Si une méthode attend un objet de type `Animal` :

```
public void faireCrier(Animal animal) {
    animal.crier();
}
```

On doit pouvoir lui passer n'importe quelle sous-classe :

```
faireCrier(new Chien("Rex"));
```

Sans provoquer d'erreur logique ou de comportement incohérent.

Violation du principe

Supposons une classe mère :

```java
public class Oiseau {
    public void voler() {
        System.out.println("Je vole.");
    }
}
```

Puis une sous-classe :

```java
public class Autruche extends Oiseau {
    @Override
    public void voler() {
        throw new UnsupportedOperationException();
    }
}
```

Ici, une `Autruche` est déclarée comme étant un `Oiseau`, mais elle ne peut pas voler. Le comportement attendu est brisé.

L'héritage est ici mal modélisé.

La relation « est-un » est incorrecte du point de vue comportemental.

Règles importantes

Pour respecter le principe de substitution :

— une sous-classe ne doit pas restreindre les capacités de la classe mère ;
— elle peut spécialiser un comportement, mais pas le contredire ;
— elle ne doit pas renforcer les contraintes d'entrée ;
— elle ne doit pas affaiblir les garanties de sortie.

Conclusion

Le principe de substitution de Liskov permet de vérifier si un héritage est cohérent. S'il n'est pas respecté :

— l'héritage est mal choisi ;
— il faut préférer la composition ;
— ou revoir la hiérarchie des classes.

8.8 Bonnes pratiques

— Ne pas utiliser l'héritage uniquement pour partager du code.
— Préférer la composition si la relation n'est pas clairement « est-un ».

— Garder les attributs `private`.
— Utiliser systématiquement `@Override`.
— Respecter la cohérence métier.

8.9 Exercices d'application

Exercice 8.1 : Hiérarchie et polymorphisme

On souhaite modéliser différents types d'employés dans une entreprise.

Travail demandé :

- créer une classe abstraite `Employe` contenant :
  - un attribut `nom` ;
  - un constructeur passant par un setter ;
  - une méthode abstraite `calculerSalaire()` ;
- créer deux sous-classes :
  - `EmployeeTempsPlein` ;
  - `EmployeFreelance` ;
- redéfinir la méthode `calculerSalaire()` dans chaque sous-classe selon les règles suivantes :
  - pour un `EmployeTempsPlein`, le salaire correspond à un montant fixe mensuel ;
  - pour un `EmployeFreelance`, le salaire correspond au nombre d'heures travaillées multiplié par un taux horaire.
- utiliser `@Override` ;

Dans une classe `Main` :

- créer un tableau de type `Employe` ;
- y placer différents types d'employés ;
- afficher le salaire de chacun.

Exercice 8.2 : Analyse d'une hiérarchie incorrecte

On considère la hiérarchie suivante :

```java
public class Rectangle {

    protected int largeur;
    protected int hauteur;

    public void setLargeur(int largeur) {
        this.largeur = largeur;
    }

    public void setHauteur(int hauteur) {
        this.hauteur = hauteur;
    }
```

```java

    public int calculerSurface() {
        return largeur * hauteur;
    }
}

public class Carre extends Rectangle {

    @Override
    public void setLargeur(int largeur) {
        this.largeur = largeur;
        this.hauteur = largeur;
    }

    @Override
    public void setHauteur(int hauteur) {
        this.largeur = hauteur;
        this.hauteur = hauteur;
    }
}
```

Travail demandé :

— Expliquer pourquoi cette hiérarchie peut violer le principe de substitution de Liskov ;

— Illustrer le problème avec un exemple de code ;

Bonus : Proposer une modélisation alternative plus cohérente.Pour résoudre cela vous aurez sans doute besoin des interfaces. Voir le chapitre suivant

Chapitre 9

Interfaces, composition

9.1 Pourquoi les interfaces ?

L'héritage permet de spécialiser un comportement. Mais il présente une limite importante : **Une classe ne peut hériter que d'une seule classe.**

En revanche, une classe peut implémenter plusieurs interfaces.

Une interface n'est pas une relation « est-un »

Une interface ne modélise pas une spécialisation.

Elle ne dit pas qu'un objet « est » quelque chose.

Elle indique qu'un objet **possède une capacité** ou qu'il **respecte un contrat**.

Exemple :

```
public interface Payable {
    double calculerSalaire();
}
```

Une classe qui implémente cette interface :

```
public class Employe implements Payable {
    @Override
    public double calculerSalaire() {
        return 2500;
    }
}
```

Ici :

— un `Employe` n'est pas un `Payable` ;

— un `Employe` est un objet qui peut être payé ;

— il respecte le contrat `Payable`.

Une interface définit ce que l'on doit faire, pas comment on le fait.

Rôle des interfaces

Les interfaces permettent de :

- définir un **contrat de comportement** ;
- découpler les implémentations ;
- favoriser le polymorphisme ;
- réduire le couplage ;
- modéliser des capacités transversales.

Exemple de polymorphisme via interface :

Tout objet vendable doit pouvoir fournir son prix et permettre sa modification [1].Supposons que l'on souhaite modéliser des éléments pouvant être vendus.

```java
public interface Vendable {
    double getPrix();
}
```

Différentes classes peuvent implémenter cette interface.

Classe Boisson :

```java
public class Boisson implements Vendable {

    private double prix;

    public Boisson(double prix) {
        setPrix(prix); // Passage obligatoire par le setter
    }

    public void setPrix(double prix) {
        if (prix >= 0) {
            this.prix = prix;
        }
    }

    @Override
    public double getPrix() {
        return this.prix;
    }
}
```

1. Dans certains modèles métier, le prix d'un objet peut être calculé dynamiquement (par exemple à partir de promotions, de règles tarifaires ou d'un service externe). Dans ce cas, il peut être préférable de ne définir dans l'interface que la méthode `getPrix()`, et de gérer la modification du prix par d'autres mécanismes. Le choix dépend de la conception du modèle et des règles métier de l'application.

Classe TeeShirt :

```java
public class TeeShirt implements Vendable {

    private double prix;

    public TeeShirt(double prix) {
        setPrix(prix); // Passage obligatoire par le setter
    }

    public void setPrix(double prix) {
        if (prix >= 0) {
            this.prix = prix;
        }
    }

    @Override
    public double getPrix() {
        return this.prix;
    }
}
```

On peut alors écrire :

```java
public void afficherPrix(Vendable v) {
    System.out.println(v.getPrix());
}
```

Cette méthode fonctionne avec :

— une `Boisson` ;

— un `TeeShirt` ;

— tout autre objet implémentant `Vendable`.

L'interface modélise ici une capacité : être vendable.

L'interface introduit un polymorphisme basé sur le contrat plutôt que sur la hiérarchie.

9.2 Définition d'une interface

```java
public interface Payable {
    double calculerSalaire();
}
```

Une interface :

— ne contient pas d'attributs d'instance ;

— ne contient que des méthodes abstraites (par défaut) ;

— définit un contrat.

Rappel : méthodes abstraites

Une **méthode abstraite** ne possède pas d'implémentation. Elle définit uniquement une signature que les classes concrètes devront obligatoirement implémenter.

Contrairement aux méthodes classiques, une méthode abstraite :

— ne possède **aucun corps** ;

— ne contient **pas d'accolades** ;

— se termine par un point-virgule.

La classe abstraite impose donc un comportement que toutes les sous-classes devront implémenter.

Attention :

La déclaration suivante n'est **pas** une méthode abstraite :

```
public void crier() { }
```

Il s'agit d'une méthode concrète possédant simplement un corps vide.

9.3 Implémentation d'une interface

```
public class EmployeTempsPlein implements Payable {

    private double salaireMensuel;

    public EmployeTempsPlein(double salaireMensuel) {
        this.salaireMensuel = salaireMensuel;
    }

    @Override
    public double calculerSalaire() {
        return this.salaireMensuel;
    }
}
```

Une classe peut implémenter plusieurs interfaces :

```
public class Consultant implements Payable, Serializable {
}
```

9.4 Introduction à la composition

Nous avons vu au chapitre précédent que l'héritage doit respecter la relation « est-un ». Si ce n'est pas le cas, il faut utiliser la **composition**.

Relation « a-un »

```
public class Voiture {

    private Moteur moteur;

    public Voiture(Moteur moteur) {
        this.moteur = moteur;
    }
}
```

Ici :
- une voiture n'est pas un moteur ;
- une voiture possède un moteur ;
- on modélise une relation « a-un ».

La composition favorise la flexibilité.

9.5 Favoriser la composition

Pourquoi préférer la composition ?
- elle réduit le couplage ;
- elle évite des hiérarchies artificielles ;
- elle facilite les modifications ;
- elle respecte mieux le principe de substitution.

Exemple : modélisation d'une voiture

Supposons que l'on souhaite modéliser une voiture possédant un moteur.
Une première tentative incorrecte pourrait être :

```
public class Voiture extends Moteur {

}
```

Cette modélisation est incorrecte.
- Une voiture **n'est pas** un moteur ;

— une voiture **possède** un moteur.

Il ne s'agit donc pas d'une relation « est-un », mais d'une relation « a-un ».

La modélisation correcte utilise la composition :

```
public class Voiture {

    private Moteur moteur;

    public Voiture(Moteur moteur) {
        this.moteur = moteur;
    }
}
```

Pourquoi est-ce plus flexible ?

Avec cette approche :

— on peut changer le type de moteur ;

— on peut remplacer le moteur sans modifier la classe `Voiture` ;

— on peut utiliser différents moteurs (diesel, électrique, hybride).

Par exemple :

```
Moteur moteurDiesel = new MoteurDiesel();
Voiture voiture = new Voiture(moteurDiesel);
```

Puis plus tard :

```
Moteur moteurElectrique = new MoteurElectrique();
Voiture voiture = new Voiture(moteurElectrique);
```

La classe `Voiture` ne change pas.

Le comportement peut varier sans modifier la classe principale.

C'est ce qui rend la composition plus flexible que l'héritage dans de nombreux cas.

On dit souvent :

« Favor composition over inheritance ». [2]

2. Littéralement : « privilégier la composition plutôt que l'héritage ». Cette recommandation est largement utilisée en conception orientée objet car la composition offre généralement une architecture plus flexible et moins couplée que l'héritage.

9.6 Le pattern Strategy

Dans de nombreux programmes, il est nécessaire de choisir entre plusieurs algorithmes ou comportements.

Une première approche consiste souvent à utiliser plusieurs conditions :

```java
public void payer(String type, double montant) {

    if (type.equals("carte")) {
        System.out.println("Paiement par carte");
    }
    else if (type.equals("paypal")) {
        System.out.println("Paiement Paypal");
    }
    else if (type.equals("crypto")) {
        System.out.println("Paiement crypto");
    }

}
```

Cette solution fonctionne, mais elle présente plusieurs inconvénients :

— le code devient difficile à maintenir ;
— chaque nouveau comportement nécessite de modifier la classe ;
— le nombre de conditions peut rapidement augmenter.

Pour résoudre ce problème, on peut utiliser le **pattern Strategy**.

9.6.1 Principe

Le pattern Strategy consiste à :

— définir une **interface** représentant un comportement ;
— créer plusieurs **implémentations** de cette interface ;
— permettre à une classe d'utiliser dynamiquement l'une de ces implémentations.

Chaque implémentation représente alors une **stratégie** différente.

9.6.2 Structure

Le pattern Strategy repose généralement sur trois éléments :

— une interface représentant la stratégie ;
— plusieurs classes implémentant cette stratégie ;
— une classe *contexte* utilisant la stratégie.

Strategy (interface)

↑

Implémentations
↑
Classe contexte

9.6.3 Exemple

On souhaite modéliser différents modes de paiement.

```
public interface StrategiePaiement {
    void payer(double montant);
}
```

Deux implémentations possibles :

```
public class PaiementCarte implements StrategiePaiement {

    @Override
    public void payer(double montant) {
        System.out.println("Paiement par carte : " + montant);
    }

}
```

```
public class PaiementPortefeuille implements StrategiePaiement {

    @Override
    public void payer(double montant) {
        System.out.println("Paiement via portefeuille : " +
            montant);
    }

}
```

Une classe utilisant ces stratégies :

```
public class Checkout {

    private StrategiePaiement strategie;

    public void setStrategie(StrategiePaiement strategie) {
        this.strategie = strategie;
    }

    public void payer(double montant) {
        strategie.payer(montant);
```

```
    }

}
```

Utilisation :

```
Checkout c = new Checkout();

c.setStrategie(new PaiementCarte());
c.payer(50);

c.setStrategie(new PaiementPortefeuille());
c.payer(20);
```

9.6.4 Avantages

Le pattern Strategy présente plusieurs avantages :

— réduire l'utilisation de nombreuses conditions ;
— faciliter l'ajout de nouveaux comportements ;
— respecter le principe *Open/Closed* ;
— découpler les algorithmes du code principal.

9.6.5 Lien avec les interfaces

Le pattern Strategy est très souvent utilisé avec les interfaces. Chaque implémentation représente un comportement interchangeable.

Dans les exercices suivants, vous allez utiliser ce principe pour créer des règles de calcul de prix applicables à un produit.

9.7 Introduction aux principes SOLID

Les principes SOLID sont cinq règles de conception orientée objet visant à produire un code maintenable et évolutif.

— **S** : Single Responsibility Principle ;
— **O** : Open/Closed Principle ;
— **L** : Liskov Substitution Principle ;
— **I** : Interface Segregation Principle ;
— **D** : Dependency Inversion Principle.

S – Single Responsibility

Une classe ne devrait avoir qu'une seule responsabilité, et donc une seule raison d'être modifiée.

O – Open/Closed

Le principe **Open/Closed** stipule que :

Une classe doit être ouverte à l'extension, mais fermée à la modification.

Cela signifie que l'on doit pouvoir ajouter de nouveaux comportements sans modifier le code existant.

Problème sans ce principe

Supposons une application qui calcule le prix de différents produits :

```java
public double calculerPrix(Object produit) {

    if (produit instanceof Livre) {
        return ((Livre) produit).getPrix();
    }

    if (produit instanceof TeeShirt) {
        return ((TeeShirt) produit).getPrix();
    }

    return 0;
}
```

[3]

Si l'on ajoute un nouveau type de produit, par exemple `Boisson`, il faudra modifier cette méthode pour ajouter un nouveau cas.

Le code doit donc être modifié à chaque évolution.

Cela viole le principe Open/Closed.

Application du principe

On peut utiliser une interface :

3. L'opérateur `instanceof` permet de vérifier si un objet est une instance d'une classe donnée ou d'une de ses sous-classes. Son utilisation excessive est souvent un signe d'une mauvaise conception orientée objet, car elle remplace parfois le polymorphisme par des tests de type.

```java
public interface Vendable {
    double getPrix();
}
```

Chaque produit implémente alors ce comportement :

```java
public class Livre implements Vendable {
    public double getPrix() {
        return 15.0;
    }
}
```

La méthode devient :

```java
public double calculerPrix(Vendable produit) {
    return produit.getPrix();
}
```

Avantage

Si l'on ajoute une nouvelle classe :

```java
public class Boisson implements Vendable {
    public double getPrix() {
        return 2.5;
    }
}
```

Aucune modification n'est nécessaire dans le code existant.

Le système est étendu sans modifier les classes existantes.

L – Liskov

Vu au chapitre précédent.

I – Interface Segregation

Il vaut mieux plusieurs petites interfaces spécialisées qu'une grosse interface générale.

D – Dependency Inversion

Le principe de **Dependency Inversion** (inversion des dépendances) stipule que :

Les classes importantes du système ne doivent pas dépendre directement des implémentations concrètes, mais d'abstractions.

Autrement dit, une classe ne doit pas créer elle-même les objets dont elle dépend. Ces dépendances doivent être fournies depuis l'extérieur du composant.

Première approche (couplage fort)

Dans une première approche, une classe peut créer directement l'objet dont elle a besoin.

```java
public class FactureService {

    private MySQLDatabase database = new MySQLDatabase();

    public void enregistrer() {
        database.save();
    }
}
```

Cette solution fonctionne, mais elle présente un problème important :

— la classe `FactureService` dépend directement de `MySQLDatabase` ;
— si l'on souhaite changer de base de données, il faudra modifier cette classe ;
— le code devient fortement couplé.

Par exemple, si l'on souhaite utiliser une autre base de données :

```
PostgreSQLDatabase
```

il faudra modifier le code existant.

Application du principe

Pour réduire ce couplage, on introduit une abstraction.

```java
public interface Database {
    void save();
}
```

Une implémentation concrète peut être :

```java
public class MySQLDatabase implements Database {

    @Override
    public void save() {
        System.out.println("Sauvegarde dans MySQL");
    }
}
```

La classe `FactureService` ne dépend plus d'une implémentation précise, mais de l'interface :

```java
public class FactureService {

    private Database database;

    public FactureService(Database database) {
        this.database = database;
    }

    public void enregistrer() {
        database.save();
    }
}
```

Utilisation dans la classe principale

L'objet concret est maintenant créé à l'extérieur de la classe.

```java
public class Main {

    public static void main(String[] args) {
        Database db = new MySQLDatabase();
        FactureService service = new FactureService(db);
        service.enregistrer();
    }
}
```

Avantages

- la classe `FactureService` ne dépend plus d'une base de données spécifique ;
- l'implémentation peut être remplacée facilement ;
- le système devient plus flexible et plus maintenable.

Par exemple, il suffit de modifier la ligne suivante :

```java
Database db = new PostgreSQLDatabase();
```

sans modifier la classe `FactureService`.

Le système dépend désormais d'une abstraction plutôt que d'une implémentation concrète.

Conclusion

Les interfaces et la composition permettent de concevoir des architectures plus souples que l'héritage seul.

Elles constituent le socle des architectures modernes :

— frameworks ;

— injection de dépendances ;

— architectures en couches ;

— développement JavaFX.

Dans le chapitre suivant, nous mettrons en pratique ces principes dans une architecture applicative plus complète.

Exercices d'application

Exercice 9.1 : Première interface

On souhaite modéliser différents types de produits pouvant être vendus.

Créer une interface `Vendable` contenant la méthode suivante :

— `double getPrix()`

Créer ensuite deux classes :

— `Livre`

— `Boisson`

Contraintes :

— les deux classes doivent implémenter l'interface `Vendable` ;

— chaque classe doit contenir un attribut `prix` ;

— utiliser un constructeur pour initialiser le prix ;

— le constructeur doit passer par le setter.

Dans une classe `Main` :

— créer un `Livre` et une `Boisson` ;

— afficher leur prix en appelant la méthode `getPrix()`.

Exercice 9.2 : Application du principe Dependency Inversion

On souhaite créer une application capable d'envoyer des notifications.

Dans une première version du programme, la classe suivante est utilisée :

```java
public class NotificationService {

    private EmailService email = new EmailService();

    public void envoyer(String message) {
        email.send(message);
    }
}
```

Cette conception crée un couplage fort entre `NotificationService` et `EmailService`.

Travail demandé :

1. Créer une interface `MessageService` contenant la méthode :
   — `void send(String message)`
2. Modifier la classe `EmailService` pour qu'elle implémente cette interface.
3. Créer une seconde implémentation :

— `SMSService`

4. Modifier la classe `NotificationService` afin qu'elle dépende de l'interface `MessageService` plutôt que d'une classe concrète.
5. Utiliser un constructeur pour recevoir la dépendance.

Dans la classe `Main` :
— créer un `EmailService` ;
— créer un `NotificationService` en lui passant cette implémentation ;
— envoyer un message.
Tester ensuite le programme en remplaçant `EmailService` par `SMSService`.

Exercice 9.3 : Stratégies de calcul de prix

On souhaite créer un système permettant d'appliquer différentes règles de calcul sur un prix.

L'objectif est de concevoir une architecture flexible permettant d'ajouter facilement de nouvelles règles de calcul.

Travail demandé :

1. Créer une interface `ReglePrix` contenant la méthode :

   ```
   double appliquer(double prix)
   ```

2. Ajouter dans cette interface une méthode par défaut :

   ```
   default double limiter(double valeur, double min, double max)
   ```

   Cette méthode doit limiter une valeur entre un minimum et un maximum.

3. Créer les implémentations suivantes :
   — `RemiseFixe` : applique une réduction fixe ;
   — `RemisePourcentage` : applique une réduction en pourcentage avec un plafond ;
   — `TVA` : applique une taxe sur le prix.

4. Créer une classe abstraite `PipelinePrix` contenant une méthode :

   ```
   double appliquerToutes(List<ReglePrix> regles, double prix)
   ```

   Cette méthode doit appliquer successivement toutes les règles de prix.

5. Tester le système dans une classe `Main` :
   — créer plusieurs règles ;
   — les stocker dans une `List<ReglePrix>` ;
   — appliquer toutes les règles sur un prix initial.

Chapitre 10

Collections et généricité

10.1 Limites des tableaux

Jusqu'à présent, nous avons utilisé des tableaux :

```
int[] notes = {12, 8, 15, 3};
```

Les tableaux présentent plusieurs limitations :

— leur taille est fixe ;

— ils ne possèdent que peu de fonctionnalités ;

— ils ne permettent pas facilement d'ajouter ou supprimer des éléments.

Par exemple, un tableau ne peut pas changer de taille :

```
int[] notes = new int[10];
```

Si le tableau est plein, il faut en créer un nouveau.

Pour résoudre ces limitations, Java propose le **Framework de Collections**.

Quelques précisions importantes

les tableaux sont des objets

En Java, un tableau est en réalité un **objet**.

Même si la syntaxe des tableaux peut rappeler celle des langages procéduraux comme le C, un tableau est instancié comme n'importe quel objet Java et est stocké en mémoire sur le *heap*.[1]

Par exemple :

```
int[] notes = new int[10];
```

1. Le *heap* (tas mémoire) est la zone de mémoire dans laquelle les objets Java sont créés pendant l'exécution du programme. Lorsqu'on instancie un objet avec le mot-clé `new`, celui-ci est généralement alloué dans le *heap*. La gestion de cette mémoire est assurée automatiquement par la machine virtuelle Java grâce au *garbage collector*, qui libère les objets devenus inutilisés.

Cette instruction crée un objet tableau capable de contenir 10 entiers.

Un tableau possède donc certaines propriétés, notamment l'attribut `length` permettant de connaître sa taille :

```
System.out.println(notes.length);
```

La classe `Object`

En Java, toutes les classes héritent implicitement d'une classe fondamentale appelée `Object`.

`Object` est la classe racine de toute la hiérarchie Java.

Cela signifie que tous les objets Java possèdent les méthodes définies dans cette classe.

Par exemple :

- `toString()` : représentation textuelle de l'objet ;
- `equals()` : comparaison entre deux objets ;
- `hashCode()` : identifiant utilisé notamment dans les collections.

Même les tableaux héritent de cette classe.

Ainsi, on peut écrire :

```
int[] notes = new int[10];

System.out.println(notes.toString());
```

car les tableaux possèdent également les méthodes héritées de la classe `Object`.

Cas particulier des chaînes de caractères

Les chaînes de caractères en Java sont également des objets. Elles appartiennent à la classe `String`.

Cependant, Java propose une syntaxe simplifiée permettant de créer ces objets de manière implicite.

Par exemple :

```
String nom = "Alice";
```

En réalité, cette instruction crée un objet de type `String`. Java instancie cet objet automatiquement.

L'écriture précédente est équivalente à :

```
String nom = new String("Alice");
```

Cependant, cette seconde écriture est rarement utilisée, car la première est plus simple et plus efficace.

Les objets `String` possèdent de nombreuses méthodes utiles, par exemple :

- `length()` : retourne la longueur de la chaîne ;
- `toUpperCase()` : convertit la chaîne en majuscules ;
- `substring()` : extrait une partie de la chaîne.

Exemple :

```
String nom = "Alice";

System.out.println(nom.length());
System.out.println(nom.toUpperCase());
```

10.2 Le framework de collections

Le framework de collections fournit des structures de données dynamiques permettant de manipuler facilement des groupes d'objets.

Les collections les plus utilisées sont :

- **List**
- **Set**
- **Map**

Une collection est une structure permettant de stocker et manipuler un ensemble d'objets.

10.3 La collection List

Une **List** est une collection ordonnée d'éléments.

Les éléments sont accessibles par index, comme dans un tableau.

L'implémentation la plus utilisée est :

ArrayList

Création d'une liste

```
import java.util.ArrayList;
import java.util.List;

List<String> noms = new ArrayList<>();
```

Ajouter des éléments

```
noms.add("Alice");
noms.add("Bob");
noms.add("Charlie");
```

Accéder à un élément

```
System.out.println(noms.get(0));
```

Supprimer un élément

```
noms.remove("Bob");
```

10.4 Parcourir une collection

Plusieurs méthodes permettent de parcourir une collection.

Boucle classique

```
for (int i = 0; i < noms.size(); i++) {
    System.out.println(noms.get(i));
}
```

Boucle améliorée

```
for (String nom : noms) {
    System.out.println(nom);
}
```

10.5 Collections d'objets

Les collections peuvent contenir des objets.

```
List<Etudiant> etudiants = new ArrayList<>();
```

On peut ensuite ajouter des objets :

```
etudiants.add(new Etudiant("Durand", 18));
etudiants.add(new Etudiant("Martin", 20));
```

Parcours :

```
for (Etudiant e : etudiants) {
    System.out.println(e.getNom());
}
```

10.6 Pourquoi utiliser des génériques ?

Considérons la collection suivante :

```
ArrayList liste = new ArrayList();
```

On peut y ajouter n'importe quoi :

```
liste.add("Bonjour");
liste.add(42);
liste.add(new Etudiant());
```

Cela peut provoquer des erreurs à l'exécution.

Les **génériques** permettent de spécifier le type d'élément contenu dans la collection.

```
List<String> noms = new ArrayList<>();
```

Ici, la liste ne peut contenir que des objets `String`.

10.7 Avantages des génériques

- vérification des types à la compilation ;
- code plus sûr ;
- suppression des conversions de type.

Exemple sans génériques :

```
String nom = (String) liste.get(0);
```

Avec génériques :

```
String nom = noms.get(0);
```

10.8 Set

Un **Set** représente un ensemble d'éléments **uniques**. Cela signifie qu'un même élément ne peut apparaître qu'une seule fois dans la collection.

```
import java.util.Set;
import java.util.HashSet;

```

```
4 Set<String> noms = new HashSet<>();
5
6 noms.add("Alice");
7 noms.add("Bob");
8 noms.add("Alice"); // ignoré
```

Dans cet exemple, la seconde insertion de `"Alice"` n'est pas prise en compte car un `Set` ne peut contenir de doublons.

Quand utiliser un Set ?

Un `Set` est utile lorsque l'on souhaite :

— stocker des éléments sans duplication ;
— vérifier rapidement si un élément est déjà présent ;
— représenter un ensemble d'éléments uniques.

Exemples d'utilisation :

— une liste d'adresses e-mail d'utilisateurs ;
— un ensemble de mots dans un dictionnaire ;
— des identifiants uniques ;
— les tags associés à un article.

Intérêt par rapport à une List

Contrairement à une `List` :

— un `Set` ne garantit pas l'ordre des éléments ;
— il empêche automatiquement les doublons ;
— il permet de vérifier rapidement si un élément existe déjà.

Ainsi, si l'on souhaite simplement stocker une collection d'éléments uniques sans se soucier de leur position, le `Set` est souvent plus adapté qu'une `List`.

Principales implémentations de Set

Il existe plusieurs implémentations de l'interface `Set`. Les plus courantes sont :

HashSet

`HashSet` est l'implémentation la plus utilisée.

— les éléments sont uniques ;
— l'ordre d'insertion n'est pas garanti ;
— les opérations d'ajout et de recherche sont généralement très rapides.

Exemple :

```java
Set<String> noms = new HashSet<>();

noms.add("Alice");
noms.add("Bob");
noms.add("Charlie");
```

LinkedHashSet

`LinkedHashSet` conserve l'ordre d'insertion des éléments.

— les éléments restent uniques ;
— l'ordre dans lequel les éléments ont été ajoutés est conservé ;
— légèrement moins performant que `HashSet`.

Exemple :

```java
Set<String> noms = new LinkedHashSet<>();

noms.add("Alice");
noms.add("Bob");
noms.add("Charlie");
```

TreeSet

`TreeSet` stocke les éléments dans un ordre trié.

— les éléments sont automatiquement triés ;
— les doublons sont interdits ;
— les opérations sont généralement plus lentes que dans `HashSet`.

Exemple :

```java
Set<Integer> nombres = new TreeSet<>();

nombres.add(5);
nombres.add(2);
nombres.add(8);

System.out.println(nombres); // [2, 5, 8]
```

Map

Une **Map** permet d'associer une **clé** à une **valeur**. On parle souvent d'association *clé – valeur*.

Chaque clé est unique et permet d'accéder rapidement à la valeur correspondante.

```java
import java.util.Map;
import java.util.HashMap;

Map<String, Integer> notes = new HashMap<>();

notes.put("Alice", 14);
notes.put("Bob", 17);
```

Dans cet exemple :

— la clé est une chaîne de caractères représentant le nom ;

— la valeur est un entier représentant la note.

Accéder à une valeur

Pour récupérer la valeur associée à une clé, on utilise la méthode `get()` :

```java
System.out.println(notes.get("Alice"));
```

Quand utiliser une Map ?

Une `Map` est utile lorsque l'on souhaite retrouver rapidement une information à partir d'une clé.

Exemples d'utilisation :

— associer un nom d'utilisateur à un mot de passe ;

— associer un identifiant à un objet ;

— stocker des paramètres de configuration ;

— associer un produit à son prix.

Différence avec une List

Contrairement à une `List` :

— on n'accède pas aux éléments par index ;

— chaque valeur est associée à une clé ;

— les clés doivent être uniques.

Principales implémentations de Map

Comme pour les `Set`, plusieurs implémentations existent.

HashMap

`HashMap` est l'implémentation la plus utilisée.

— les clés sont uniques ;
— l'ordre des éléments n'est pas garanti ;
— les opérations d'accès sont généralement très rapides.

LinkedHashMap

`LinkedHashMap` conserve l'ordre d'insertion des éléments.
— les clés restent uniques ;
— l'ordre dans lequel les éléments ont été ajoutés est conservé.

TreeMap

`TreeMap` stocke les éléments dans un ordre trié selon les clés.
— les clés sont triées automatiquement ;
— les opérations sont généralement plus lentes que dans `HashMap`.

Résumé

— une `Map` associe une clé à une valeur ;
— chaque clé est unique ;
— `HashMap` est l'implémentation la plus utilisée ;
— les `Map` permettent d'accéder rapidement à une valeur à partir d'une clé.

10.9 Intérêt des collections

— Les tableaux ont une taille fixe.
— Les collections permettent de manipuler des ensembles dynamiques d'objets.
— `List` est la collection la plus utilisée.
— Les génériques permettent de sécuriser les types.

Les collections sont omniprésentes dans les applications Java et constituent une compétence essentielle pour tout développeur.

Collections et polymorphisme

Les collections deviennent particulièrement intéressantes lorsqu'elles sont utilisées avec l'héritage et le polymorphisme.

Elles permettent de stocker dans une même collection des objets de types différents, à condition qu'ils partagent une classe mère ou une interface commune.

Considérons une classe abstraite `Animal` :

```
1 public abstract class Animal {
2
```

```
    private String nom;

    public Animal(String nom) {
        setNom(nom);
    }

    public String getNom() {
        return this.nom;
    }

    public void setNom(String nom) {
        if (nom != null && !nom.isEmpty()) {
            this.nom = nom;
        }
    }

    public abstract void crier();
}
```

Nous pouvons ensuite créer plusieurs classes héritant de cette classe.

```
public class Chien extends Animal {

    public Chien(String nom) {
        super(nom);
    }

    @Override
    public void crier() {
        System.out.println("Le chien aboie");
    }
}
```

```
public class Chat extends Animal {
    public Chat(String nom) {
        super(nom);
    }

    @Override
    public void crier() {
        System.out.println("Le chat miaule");
    }
}
```

Grâce au polymorphisme, il est possible de stocker différents types d'animaux dans une même collection.

```
List<Animal> animaux = new ArrayList<>();
animaux.add(new Chien("Rex"));
animaux.add(new Chat("Minou"));
```

Nous pouvons ensuite parcourir la collection :

```
for (Animal animal : animaux) {
    animal.crier();
}
```

Lors de l'exécution, chaque objet utilisera sa propre implémentation de la méthode `crier()`.

Une collection peut contenir différents objets tant qu'ils partagent un type commun.

Cela illustre l'un des principes fondamentaux de la programmation orientée objet : le polymorphisme.

Un même code peut manipuler différents objets via leur classe mère ou leur interface commune.

10.10 Comparer et trier des objets

Lorsque l'on manipule des objets dans des collections, il est souvent nécessaire de :

— comparer deux objets ;
— vérifier si deux objets sont égaux ;
— trier une liste d'objets.

Java fournit plusieurs mécanismes pour réaliser ces opérations :

— la méthode `equals()` ;
— la méthode `hashCode()` ;
— l'interface `Comparable` ;
— l'interface `Comparator`.

Égalité des objets

En Java, deux objets peuvent être comparés de deux façons différentes.

Comparaison avec ==

L'opérateur == compare les **références** des objets.

```
Etudiant e1 = new Etudiant("Alice");
Etudiant e2 = new Etudiant("Alice");

System.out.println(e1 == e2);
```

Résultat :

```
false
```

Même si les deux étudiants ont le même nom, ils sont stockés dans des objets différents en mémoire.

Comparaison avec `equals()`

La méthode `equals()` permet de définir une **égalité logique** entre deux objets.

Par défaut, la classe `Object` définit `equals()` comme une comparaison de références, exactement comme ==.

Il est donc souvent nécessaire de redéfinir cette méthode.

Exemple

Considérons la classe suivante :

```
public class Etudiant {

    private String id;
    private String nom;

    public Etudiant(String id, String nom) {
        this.id = id;
        this.nom = nom;
    }

}
```

On peut décider que deux étudiants sont égaux s'ils possèdent le même identifiant.

```
@Override
public boolean equals(Object obj) {

    if (this == obj) {
        return true;
```

```
    }

    if (obj == null || getClass() != obj.getClass()) {
        return false;
    }

    Etudiant autre = (Etudiant) obj;

    return id.equals(autre.id);
}
```

La méthode `hashCode()`

Lorsque l'on redéfinit `equals()`, il est indispensable de redéfinir également la méthode `hashCode()`.

Cette méthode est utilisée par certaines collections comme :

- `HashSet`
- `HashMap`
- `HashTable`

Elle permet de placer les objets dans des structures de hachage afin d'accélérer les recherches.

Règle importante

Deux objets égaux selon `equals()` doivent obligatoirement avoir le même `hashCode()`.

Exemple

```
@Override
public int hashCode() {
    return id.hashCode();
}
```

Pourquoi est-ce important ?

Considérons l'exemple suivant :

```
Set<Etudiant> etudiants = new HashSet<>();

etudiants.add(new Etudiant("E1", "Alice"));
etudiants.add(new Etudiant("E1", "Alice"));

```

```
System.out.println(etudiants.size());
```

Si `equals()` et `hashCode()` sont correctement définis, le résultat sera :

```
1
```

Sinon, la collection pourrait considérer les deux objets comme différents.

10.10.1 Ordre naturel avec `Comparable`

Il est parfois nécessaire de trier des objets.

Par exemple :

— trier des étudiants par moyenne ;

— trier des produits par prix ;

— trier des utilisateurs par nom.

Pour définir un ordre naturel, une classe peut implémenter l'interface `Comparable`.

```
public class Etudiant implements Comparable<Etudiant> {

    private String nom;
    private double moyenne;

    @Override
    public int compareTo(Etudiant autre) {
        return Double.compare(this.moyenne, autre.moyenne);
    }
}
```

La méthode `compareTo()` doit retourner :

— un nombre négatif si l'objet est plus petit ;

— zéro si les objets sont égaux ;

— un nombre positif si l'objet est plus grand.

On peut ensuite trier une liste :

```
Collections.sort(listeEtudiants);
```

10.10.2 Ordres multiples avec `Comparator`

Il arrive que l'on souhaite trier les objets selon différents critères.

Par exemple :

— par nom ;

— par moyenne ;

— par âge.

Dans ce cas, on utilise l'interface `Comparator`.

```
Comparator<Etudiant> comparateurNom =
    Comparator.comparing(Etudiant::getNom);
```

Puis :

```
listeEtudiants.sort(comparateurNom);
```

10.10.3 Lien avec les collections triées

Certaines collections utilisent automatiquement un ordre de tri :

— `TreeSet`

— `TreeMap`

Ces collections utilisent soit :

— l'ordre naturel défini par `Comparable` ;

— un `Comparator`.

Il est donc important que l'ordre défini soit cohérent avec la notion d'égalité.

À retenir

— `equals()` définit l'égalité logique entre deux objets ;

— `hashCode()` est utilisé par les structures de hachage ;

— deux objets égaux doivent avoir le même `hashCode()` ;

— `Comparable` permet de définir un ordre naturel ;

— `Comparator` permet de définir plusieurs ordres de tri.

Exercices d'application

Exercice 10.1 : Gestion d'une liste d'étudiants

On souhaite manipuler une collection d'étudiants.

Créer une classe `Etudiant` contenant :

— un attribut `nom` de type `String` ;

— un attribut `age` de type `int`.

Contraintes :

— les attributs doivent être `private` ;

— créer les getters et setters ;

— le constructeur doit passer par les setters.

Dans une classe `Main` :

— créer une `List<Etudiant>` ;

— ajouter plusieurs étudiants ;

— parcourir la collection et afficher leur nom et leur âge.

Objectifs pédagogiques :

— manipuler une `List` ;

— stocker des objets dans une collection ;

— parcourir une collection avec une boucle `for`.

Exercice 10.2 : Collection et polymorphisme

On souhaite modéliser différents types d'animaux.

Créer une classe abstraite `Animal` contenant :

— un attribut `nom` ;

— une méthode abstraite `crier()`.

Créer ensuite deux classes :

— `Chien`

— `Chat`

Ces classes doivent :

— hériter de `Animal` ;

— redéfinir la méthode `crier()`.

Dans une classe `Main` :

— créer une `List<Animal>` ;

— ajouter plusieurs objets `Chien` et `Chat` ;

— parcourir la collection ;

— appeler la méthode `crier()` pour chaque animal.

Exercice 10.3 : Égalité logique d'objets

On souhaite manipuler une collection de produits.

Créer une classe `Produit` contenant :

— un attribut `reference` de type `String` ;

— un attribut `nom` de type `String`.

Contraintes :

— les attributs doivent être `private` ;

— créer les getters et setters ;

— le constructeur doit passer par les setters ;

— redéfinir les méthodes `equals()` et `hashCode()` ;

— deux produits seront considérés comme égaux s'ils possèdent la même `reference`.

Dans une classe `Main` :

— créer un `HashSet<Produit>` ;

— ajouter plusieurs produits ;

— ajouter deux produits ayant la même référence ;

— afficher le nombre d'éléments présents dans la collection ;

— afficher les produits contenus dans la collection.

Exercice 10.4 : Comparer et trier des objets

On souhaite maintenant trier des objets de type `Produit`.

Créer une nouvelle version de la classe `Produit` contenant :

— un attribut `reference` de type `String` ;

— un attribut `nom` de type `String` ;

— un attribut `prix` de type `double`.

Contraintes :

— les attributs doivent être `private` ;

— créer les getters et setters ;

— le constructeur doit passer par les setters ;

— la classe doit implémenter l'interface `Comparable<Produit>` ;

— l'ordre naturel doit être défini sur le `prix` croissant.

Dans une classe `Main` :

— créer une `List<Produit>` ;

— ajouter plusieurs produits ;

— afficher la liste avant le tri ;

— trier la liste avec `Collections.sort()` ;

— afficher la liste après le tri.

Travail complémentaire :

— créer un `Comparator<Produit>` permettant de trier les produits par `nom` ;
— effectuer un second tri avec ce comparateur.

Chapitre 11

Gestion des exceptions et robustesse

11.1 Qu'est-ce qu'une exception ?

Lorsqu'un programme s'exécute, différentes erreurs peuvent se produire :

— division par zéro ;

— accès à un index invalide dans un tableau ;

— tentative de lecture d'un fichier inexistant ;

— saisie utilisateur incorrecte.

Ces situations provoquent ce que l'on appelle une **exception**.

Une exception est un événement anormal qui interrompt le déroulement normal du programme.

Exemple :

```
int a = 10;
int b = 0;

int resultat = a / b;
```

Ce code provoque une exception :

```
java.lang.ArithmeticException
```

Sans gestion d'erreur, le programme s'arrête immédiatement.

11.2 Le bloc try / catch

Pour éviter l'arrêt brutal du programme, Java propose un mécanisme de gestion des exceptions.

```
try {
    int resultat = a / b;
```

```
} catch (ArithmeticException e) {
    System.out.println("Division par zéro impossible.");
}
```

Fonctionnement :

— le bloc `try` contient le code susceptible de provoquer une erreur ;

— le bloc `catch` intercepte l'exception.

Ainsi, le programme peut continuer à s'exécuter.

11.3 Plusieurs blocs catch

Il est possible de gérer plusieurs types d'exceptions.

```
try {
    int[] tableau = new int[3];
    tableau[5] = 10;
}
catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index invalide.");
}
catch (Exception e) {
    System.out.println("Une erreur est survenue.");
}
```

Le bloc le plus spécifique doit toujours apparaître en premier.

11.4 Le bloc finally

Le bloc `finally` contient du code exécuté qu'il y ait eu une exception ou non.

```
try {
    System.out.println("Lecture du fichier");
}
catch (Exception e) {
    System.out.println("Erreur");
}
finally {
    System.out.println("Fin de l'opération");
}
```

Ce bloc est souvent utilisé pour :

— fermer un fichier ;

— libérer une ressource ;

— fermer une connexion.

11.5 Le mot-clé throw

Il est également possible de déclencher une exception manuellement.

```
if (age < 0) {
    throw new IllegalArgumentException("Age invalide");
}
```

Cela permet de signaler qu'une règle métier a été violée.

11.6 Exceptions vérifiées et non vérifiées

Java distingue deux grandes catégories d'exceptions.

Exceptions vérifiées

Les exceptions vérifiées doivent être gérées ou déclarées dans la signature de la méthode.

Exemple :

```
IOException
```

Exceptions non vérifiées

Ces exceptions héritent de `RuntimeException`.

Exemples :

— `NullPointerException`
— `ArithmeticException`
— `IllegalArgumentException`

Ces exceptions sont généralement liées à des erreurs de programmation.

11.7 Créer sa propre exception

Dans une application métier, il peut être utile de définir ses propres exceptions.

```
public class AgeInvalideException extends Exception {

    public AgeInvalideException(String message) {
        super(message);
    }
}
```

Utilisation :

```java
if (age < 0) {
    throw new AgeInvalideException("Age négatif interdit");
}
```

11.8 Bonnes pratiques

- ne jamais ignorer une exception ;
- utiliser des messages explicites ;
- ne pas utiliser les exceptions pour contrôler le flux normal du programme ;
- intercepter les exceptions au bon niveau.

11.9 Exercices d'application

Exercice 9.1 : Gestion d'une division

Écrire une méthode :

— nommée `diviser` ;
— recevant deux entiers en paramètres ;
— retournant le résultat de la division.

La méthode doit :

— vérifier si le deuxième nombre est égal à zéro ;
— si c'est le cas, lancer une exception.

Dans la méthode `main` :

— demander deux entiers à l'utilisateur ;
— appeler la méthode `diviser` ;
— afficher le résultat.

Contraintes :

— utiliser un bloc `try / catch` ;
— afficher un message d'erreur clair en cas de division par zéro ;
— le programme ne doit pas se terminer brutalement.

Exercice 9.2 : Création d'une exception personnalisée

Créer une exception personnalisée :

— nommée `AgeInvalideException` ;
— héritant de la classe `Exception`.

Créer ensuite une classe `Personne` contenant :

— un attribut `nom` ;
— un attribut `age`.

Dans le constructeur :

— si l'âge est négatif, lancer l'exception `AgeInvalideException`.

Dans la méthode `main` :

— demander à l'utilisateur un nom et un âge ;
— créer un objet `Personne` ;
— gérer l'exception avec un bloc `try / catch`.

Contraintes :

— l'exception doit contenir un message explicite ;
— le programme doit continuer son exécution après la gestion de l'erreur.

Exercice 9.3 : Saisie robuste et multi-catch

Écrire un programme qui :
— demande à l'utilisateur de saisir un entier `a` ;
— demande à l'utilisateur de saisir un entier `b` ;
— affiche le résultat de la division `a / b`.
Le programme doit gérer les erreurs suivantes :
— si l'utilisateur saisit autre chose qu'un entier ;
— si l'utilisateur saisit `0` pour `b`.
Dans tous les cas, le programme doit afficher :
— « Fin du programme ».
Contraintes :
— utiliser `Scanner` ;
— utiliser un bloc `try / catch` ;
— utiliser un **multi-catch** lorsque c'est pertinent ;
— utiliser un bloc `finally` pour garantir l'affichage final ;
— le bloc `try` doit être le plus court possible.

Extension : Tant que la saisie n'est pas correcte, redemander les valeurs à l'utilisateur.

Exercice 9.4 : Validation métier avec throw

Écrire une méthode :
— nommée `verifierNote` ;
— recevant une note de type `double` ;
— ne retournant rien (`void`).
La méthode doit :
— vérifier que la note est comprise entre 0 et 20 ;
— si ce n'est pas le cas, lancer une `IllegalArgumentException` avec un message explicite.
Dans la méthode `main` :
— demander à l'utilisateur de saisir une note ;
— appeler `verifierNote` ;
— si la note est valide, afficher « Note enregistrée » ;
— sinon, afficher le message d'erreur en interceptant l'exception.
Contraintes :
— utiliser `throw` dans la méthode `verifierNote` ;
— gérer l'exception avec un bloc `try / catch` dans `main` ;
— aucun affichage ne doit être réalisé dans la méthode `verifierNote`.

Chapitre 12

Entrées / Sorties et communication

12.1 Introduction

Un programme ne fonctionne généralement pas de manière isolée.

Dans la plupart des applications, il est nécessaire de :

— lire des données ;

— enregistrer des informations ;

— communiquer avec d'autres systèmes.

Ces opérations sont appelées **entrées / sorties** (ou *I/O* pour *Input / Output*).

Un programme Java peut échanger des données avec l'extérieur de plusieurs manières.

Les plus courantes sont :

— les fichiers ;

— les bases de données ;

— les API.

12.2 Lecture et écriture de fichiers

Les fichiers permettent de stocker des données de manière simple.

Ils sont souvent utilisés pour :

— enregistrer des résultats ;

— sauvegarder une configuration ;

— importer ou exporter des données.

Lecture d'un fichier texte

Java propose plusieurs classes pour lire un fichier.

L'une des plus simples est `BufferedReader`.

```
BufferedReader reader =
    new BufferedReader(new FileReader("donnees.txt"));
```

```
String ligne = reader.readLine();
System.out.println(ligne);
reader.close();
```

Dans cet exemple :

— un fichier est ouvert ;
— une ligne est lue ;
— le contenu est affiché ;
— le fichier est fermé.

Écriture dans un fichier

Il est également possible d'écrire des informations dans un fichier.

```
BufferedWriter writer =
    new BufferedWriter(new FileWriter("resultat.txt"));
writer.write("Bonjour");
writer.close();
```

Ce mécanisme permet par exemple de sauvegarder les données d'une application.

12.3 Format CSV

Le format CSV (*Comma Separated Values*) est souvent utilisé pour stocker des données structurées.

Chaque ligne représente un enregistrement et les valeurs sont séparées par un séparateur (souvent une virgule ou un point-virgule).

Exemple de fichier CSV :

```
titre;auteur
1984;George Orwell
Dune;Frank Herbert
```

Lecture simple en Java :

```
BufferedReader reader =
    new BufferedReader(new FileReader("livres.csv"));
String ligne;
while((ligne = reader.readLine()) != null) {
    System.out.println(ligne);
}

reader.close();
```

Le contenu peut ensuite être transformé en objets Java.

12.4 Bases de données

Les fichiers permettent de stocker des données de manière simple. Cependant, cette solution présente plusieurs limites :

— difficulté de recherche lorsque les données deviennent nombreuses ;
— accès concurrent difficile lorsque plusieurs utilisateurs utilisent l'application ;
— risque d'incohérence des données.

Pour gérer de grandes quantités d'informations de manière fiable, les applications utilisent généralement une **base de données**.

Une base de données est un système permettant de stocker, organiser et manipuler des données de manière structurée.

Exemples de systèmes de bases de données

Il existe de nombreux systèmes de gestion de bases de données.

Les plus utilisés sont :

— MySQL
— PostgreSQL
— SQLite
— Oracle

Ces systèmes sont appelés **SGBD** (Systèmes de Gestion de Base de Données).

Les données dans une base de données

Dans une base de données relationnelle, les informations sont organisées sous forme de **tables**.

Exemple :

Titre	Auteur
1984	George Orwell
Dune	Frank Herbert

Chaque ligne représente un enregistrement et chaque colonne correspond à un attribut.

Le langage SQL

Pour manipuler les données d'une base de données, on utilise généralement le langage **SQL** (*Structured Query Language*).

Exemple d'insertion d'un livre :

```
1 INSERT INTO livre (titre, auteur)
2 VALUES ('1984', 'George Orwell');
```

Exemple de lecture :

```
SELECT * FROM livre;
```

Accès aux bases de données en Java

En Java, la communication avec une base de données se fait généralement à l'aide de l'API **JDBC** (*Java Database Connectivity*).

JDBC permet :

— d'établir une connexion avec la base de données ;

— d'exécuter des requêtes SQL ;

— de récupérer les résultats.

Connexion à la base de données

La première étape consiste à établir une connexion.

```
Connection connection =
    DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/bibliotheque",
        "user",
        "password"
    );
```

Exécution d'une requête

Une fois la connexion établie, il est possible d'exécuter une requête SQL.

```
Statement statement = connection.createStatement();

ResultSet result = statement.executeQuery(
    "SELECT titre, auteur FROM livre"
);
```

Lecture des résultats

Les résultats peuvent ensuite être parcourus.

```
while(result.next()) {

    String titre = result.getString("titre");
    String auteur = result.getString("auteur");

    System.out.println(titre + " - " + auteur);
```

```
}
```

Résumé

L'accès à une base de données en Java suit généralement les étapes suivantes :

1. établir une connexion avec la base de données ;
2. exécuter une requête SQL ;
3. récupérer les résultats.

12.4.1 Ajout du driver JDBC MySQL dans Eclipse

Pour qu'une application Java puisse communiquer avec une base de données MySQL, il est nécessaire d'ajouter un **driver JDBC**.

Un driver JDBC est une bibliothèque Java qui permet à l'application de comprendre les instructions de connexion de type :

```
jdbc:mysql://localhost:3306/bibliotheque
```

Sans ce driver, Java ne sait pas comment communiquer avec MySQL et l'erreur suivante apparaît généralement :

```
java.sql.SQLException:
No suitable driver found for jdbc:mysql://...
```

Le driver MySQL Connector/J

Le driver officiel pour MySQL s'appelle :

`mysql-connector-j`

Il est distribué sous la forme d'un fichier `.jar` (Java ARchive).

Ce fichier contient les classes nécessaires pour permettre la communication entre Java et MySQL.

Téléchargement du driver

Le driver peut être téléchargé sur le site officiel de MySQL :

`https://dev.mysql.com/downloads/connector/j/`

Après téléchargement, on obtient un fichier similaire à :

`mysql-connector-j-8.x.x.jar`

Ajout du driver dans un projet Eclipse

Pour utiliser ce driver dans un projet Java :

1. faire un clic droit sur le projet ;
2. sélectionner **Build Path** ;
3. choisir **Add External Archives** ;
4. sélectionner le fichier `mysql-connector-j.jar`.

Une fois ajouté, le fichier apparaît dans :

`Referenced Libraries`

Le driver est alors disponible pour l'application.

Exemple de connexion à MySQL

Une fois le driver ajouté, la connexion peut être établie avec la classe `DriverManager` :

```
String url = "jdbc:mysql://localhost:8889/bibliotheque";
String user = "root";
String password = "root";

Connection connection =
    DriverManager.getConnection(url, user, password);
```

Si la connexion réussit, l'application peut ensuite :
— exécuter des requêtes SQL ;
— lire les résultats ;
— modifier les données.

12.5 Les API

Une **API** (*Application Programming Interface*) permet à une application de communiquer avec un autre service.

Autrement dit, une API permet à un programme d'utiliser les fonctionnalités d'un autre programme.

De nombreuses applications modernes utilisent des API.

Par exemple :
— récupérer la météo ;
— effectuer un paiement ;
— obtenir des informations depuis un service web ;
— communiquer entre plusieurs applications.

Principe de fonctionnement

Lorsqu'une application utilise une API, elle envoie une **requête** à un serveur.

Le serveur traite cette requête et renvoie une **réponse**.

Le fonctionnement peut être résumé ainsi :

$$\text{Application Java} \rightarrow \text{API} \rightarrow \text{Serveur}$$

Le protocole HTTP

La plupart des API modernes utilisent le protocole **HTTP** (le même protocole utilisé par les navigateurs web).

Les requêtes les plus courantes sont :

— **GET** : récupérer des données ;
— **POST** : envoyer des données ;
— **PUT** : modifier des données ;
— **DELETE** : supprimer des données.

Exemple d'appel HTTP :

```
GET https://api.exemple.com/livres
```

Format des données : JSON

Les API renvoient généralement les données sous forme de **JSON** (*JavaScript Object Notation*).

Exemple de réponse JSON :

```
{
  "titre": "1984",
  "auteur": "George Orwell"
}
```

Ce format est simple à lire et à manipuler dans la plupart des langages de programmation.

Appel d'une API en Java

Depuis Java, il est possible d'effectuer des requêtes HTTP grâce à la classe `HttpClient`.

```java
HttpClient client = HttpClient.newHttpClient();

HttpRequest request =
    HttpRequest.newBuilder()
        .uri(URI.create("https://api.exemple.com/livres"))
```

```
        .GET()
        .build();
```

La requête peut ensuite être envoyée :

```
HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());

String json = response.body();
```

La réponse de l'API est alors disponible sous forme de texte et peut être traitée par l'application.

Résumé

Les API permettent aux applications de communiquer entre elles.
Le fonctionnement repose généralement sur :
— le protocole HTTP ;
— des requêtes envoyées par le programme ;
— des réponses contenant des données (souvent au format JSON).

12.6 Comparaison des différentes approches

Chaque solution possède ses avantages et ses limites.

Méthode	Utilisation
Fichier	stockage simple
Base de données	stockage structuré et volumineux
API	communication entre applications

Le choix dépend du contexte et des besoins de l'application.

12.7 Exercices d'application

Exercice 12.1 : Lecture d'un fichier texte

Écrire un programme qui :
— ouvre un fichier nommé `notes.txt` ;
— lit son contenu ligne par ligne ;
— affiche chaque ligne dans la console.
Contraintes :
— utiliser `BufferedReader` et `FileReader` ;
— utiliser une boucle `while` pour lire toutes les lignes ;
— fermer correctement le fichier après lecture.

Extension : Compter le nombre de lignes présentes dans le fichier.
Exemple de contenu du fichier `notes.txt` :

```
12
15
9
18
14
7
16
```

Exercice 12.2 : Écriture dans un fichier

Écrire un programme qui :
— demande à l'utilisateur de saisir trois noms de livres ;
— enregistre ces titres dans un fichier nommé `livres.txt` ;
— chaque livre doit être écrit sur une ligne différente.
Contraintes :
— utiliser `BufferedWriter` et `FileWriter` ;
— utiliser une boucle pour saisir les livres ;
— fermer le fichier après l'écriture.

Extension : Afficher un message indiquant que les données ont été enregistrées avec succès.

Exercice 12.3 : Lecture d'un fichier CSV

On dispose du fichier `livres.csv` suivant :

```
titre;auteur
```

```
1984;George Orwell
Dune;Frank Herbert
Fondation;Isaac Asimov
```

Écrire un programme qui :

— lit le fichier `livres.csv` ;
— sépare les informations de chaque ligne ;
— affiche le titre et l'auteur de chaque livre.

Contraintes :

— utiliser `BufferedReader` ;
— utiliser la méthode `split(";")` pour séparer les colonnes ;
— ignorer la première ligne contenant les en-têtes.

Extension : Créer un objet `Livre` pour chaque ligne du fichier.

Exercice 12.4 : Comprendre l'appel d'une API

On vous fournit le code suivant :

```java
HttpClient client = HttpClient.newHttpClient();

HttpRequest request =
    HttpRequest.newBuilder()
        .uri(URI.create("https://api.exemple.com/livres"))
        .GET()
        .build();

HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());

System.out.println(response.body());
```

Questions

1. Quel est le rôle de `HttpClient` ?
2. Quelle est l'adresse de l'API utilisée dans cet exemple ?
3. Quelle méthode HTTP est utilisée ?
4. Que contient la variable `response.body()` ?

Extension : Modifier l'adresse de l'API pour appeler un autre service web public.

Exercice 12.5 : Création d'une base de données

Dans cet exercice, vous allez créer une base de données afin de stocker des informations sur des livres.

Travail demandé

À l'aide de MySQL :

1. créer une base de données nommée `bibliotheque` ;
2. créer une table `livre` contenant les colonnes suivantes :
   - `id` : identifiant du livre (entier, clé primaire, auto-incrémentée)
   - `titre` : titre du livre
   - `auteur` : auteur du livre
3. insérer au moins trois livres dans la table.

Résultat attendu

La table doit contenir par exemple :

```
id | titre        | auteur
-------------------------------
1  | 1984         | George Orwell
2  | Dune         | Frank Herbert
3  | Fondation    | Isaac Asimov
```

Extension :

Écrire une requête SQL permettant d'afficher tous les livres dont l'auteur est `George Orwell`.

Aide : exemple de connexion à MySQL en Java

Le code suivant montre comment établir une connexion avec la base de données `bibliotheque` en utilisant JDBC.

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class TestConnexion {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/bibliotheque";
        String user = "root";
        String password = "password";

```

```
13          try {
14
15              Connection connection =
16                  DriverManager.getConnection(url, user, password);
17
18              System.out.println("Connexion reussie !");
19
20              connection.close();
21
22          } catch (SQLException e) {
23              System.out.println("Erreur de connexion");
24              e.printStackTrace();
25          }
26
27      }
28
29  }
```

Chapitre 13

Expressions Lambda

13.1 Objectifs

Dans ce chapitre nous allons découvrir les **expressions lambda** introduites dans **Java 8**.

À la fin de ce chapitre vous saurez :

— comprendre le principe des expressions lambda

— utiliser une interface fonctionnelle

— écrire des fonctions anonymes

— simplifier du code avec les lambdas

Les expressions lambda sont très utilisées dans :

— les collections

— les streams

— les événements JavaFX

Ce chapitre est donc important avant d'aborder les interfaces graphiques.

13.2 Motivation

Avant Java 8, il fallait souvent écrire beaucoup de code pour définir un comportement simple.

Par exemple avec une interface :

```
interface Operation {
    int calculer(int a, int b);
}
```

Pour utiliser cette interface :

```
Operation addition = new Operation() {
    @Override
    public int calculer(int a, int b) {
```

```
        return a + b;
    }
};
```

Ce code est lourd pour une opération simple.

Les expressions lambda permettent d'écrire ce comportement de manière beaucoup plus concise.

13.3 Principe d'une expression lambda

Une expression lambda permet de définir une **fonction anonyme**.

Syntaxe générale :

```
(paramètres) -> expression
```

ou

```
(paramètres) -> {
    instructions
}
```

Exemple :

```
(a, b) -> a + b
```

Cette expression représente une fonction qui prend deux paramètres et retourne leur somme.

13.4 Interfaces fonctionnelles

Les lambdas s'utilisent avec des **interfaces fonctionnelles**.

Une interface fonctionnelle possède :

— une seule méthode abstraite

Exemple :

```
interface Operation {
    int calculer(int a, int b);
}
```

Cette interface peut être utilisée avec une expression lambda.

13.5 Premier exemple avec lambda

Sans lambda :

```
Operation addition = new Operation() {
    @Override
    public int calculer(int a, int b) {
        return a + b;
    }
};
```

Avec lambda :

```
Operation addition = (a, b) -> a + b;
```

Le code est beaucoup plus simple.

Utilisation :

```
int resultat = addition.calculer(3, 4);
System.out.println(resultat);
```

Résultat :

7

13.6 Lambda avec bloc d'instructions

Si plusieurs instructions sont nécessaires, on utilise un bloc :

```
(a, b) -> {
    int resultat = a + b;
    return resultat;
}
```

Exemple complet :

```
Operation addition = (a, b) -> {
    System.out.println("Calcul en cours...");
    return a + b;
};
```

13.7 Exemples simples

13.7.1 Lambda sans paramètre

```
() -> System.out.println("Bonjour")
```

13.7.2 Lambda avec un paramètre

```
x -> x * x
```

13.7.3 Lambda avec plusieurs paramètres

```
(a, b) -> a * b
```

13.8 Utilisation avec les collections

Les expressions lambda sont très utiles pour parcourir une collection.

Exemple :

```
ArrayList<String> noms = new ArrayList<>();

noms.add("Alice");
noms.add("Bob");
noms.add("Charlie");
```

Parcours classique :

```
for(String nom : noms) {
    System.out.println(nom);
}
```

Avec lambda :

```
noms.forEach(nom -> System.out.println(nom));
```

La méthode `forEach()` applique la fonction à chaque élément.

13.9 Références de méthodes

Lorsque la lambda appelle simplement une méthode existante, on peut utiliser une référence de méthode.

Exemple :

```
noms.forEach(System.out::println);
```

Cela est équivalent à :

```
noms.forEach(nom -> System.out.println(nom));
```

13.10 Utilisation dans les événements

Les lambdas sont très utilisées dans les interfaces graphiques.

Exemple avec JavaFX :

```
button.setOnAction(e -> {
    System.out.println("Bouton cliqué");
});
```

Cette lambda définit le comportement exécuté lorsque l'utilisateur clique sur le bouton.

Nous utiliserons cette syntaxe dans les chapitres JavaFX.

13.11 Bonnes pratiques

Quelques bonnes pratiques pour utiliser les lambdas :

— utiliser les lambdas pour les comportements courts
— éviter les blocs trop longs
— privilégier la lisibilité
— utiliser les références de méthodes lorsque c'est possible

Les lambdas permettent de simplifier le code mais ne doivent pas rendre le programme difficile à comprendre.

13.12 Exercices

13.12.1 Exercice 1

Créer une interface fonctionnelle :

```
interface Calcul {
    int calculer(int a, int b);
}
```

Créer une lambda permettant de calculer :

— l'addition
— la multiplication

13.12.2 Exercice 2

Créer une liste d'entiers :

```
ArrayList<Integer> nombres
```

Ajouter plusieurs valeurs.

Afficher les valeurs avec :

— une boucle `for`

— une lambda avec `forEach`

13.12.3 Exercice 3

Afficher les nombres d'une liste multipliés par 2 en utilisant une lambda.

13.13 À retenir

Les expressions lambda permettent de :

— définir une fonction rapidement

— simplifier l'utilisation des interfaces fonctionnelles

— écrire du code plus concis

Elles sont très utilisées dans :

— les collections

— les streams

— les événements JavaFX

Nous utiliserons régulièrement les lambdas dans les chapitres suivants consacrés à JavaFX.

Chapitre 14

Introduction à JavaFX

14.1 Objectifs

Dans ce chapitre nous allons découvrir JavaFX, la bibliothèque graphique moderne de Java.

À la fin de ce chapitre vous saurez :

— créer une application JavaFX

— comprendre les notions de **Stage**, **Scene** et **Node**

— afficher une fenêtre graphique

— ajouter un premier composant dans l'interface

Ce chapitre constitue la première étape vers la réalisation d'applications graphiques complètes.

14.2 Applications console et interfaces graphiques

Jusqu'à présent, les programmes que nous avons réalisés sont des **applications console**.

L'interaction avec l'utilisateur se fait par du texte :

```
System.out.println("Entrez votre nom : ");
Scanner scanner = new Scanner(System.in);
String nom = scanner.nextLine();
```

Ce type d'application est simple mais peu ergonomique.

La plupart des logiciels modernes utilisent une **interface graphique (GUI)** permettant d'interagir avec :

— des boutons

— des champs de saisie

— des menus

— des tableaux

— des fenêtres

Java propose plusieurs bibliothèques pour créer des interfaces graphiques. La plus moderne est **JavaFX**.

14.3 Qu'est-ce que JavaFX ?

JavaFX est une bibliothèque permettant de créer des applications graphiques modernes en Java.

Elle permet notamment de :

— créer des fenêtres
— afficher des composants graphiques
— gérer les interactions utilisateur
— organiser une interface graphique

JavaFX remplace progressivement l'ancienne bibliothèque graphique **Swing**.

14.4 Architecture d'une application JavaFX

Une application JavaFX repose sur trois concepts principaux :

— **Stage**
— **Scene**
— **Node**

14.4.1 Le Stage

Le **Stage** représente la **fenêtre principale** de l'application.

C'est la fenêtre affichée par le système d'exploitation.

Elle possède :

— un titre
— une taille
— des boutons (réduire, agrandir, fermer)

14.4.2 La Scene

La **Scene** représente le **contenu de la fenêtre**.

Elle contient tous les éléments graphiques de l'interface.

Une application possède généralement une seule scène principale.

14.4.3 Les Nodes

Les **Nodes** représentent les éléments graphiques affichés dans l'interface.

Exemples :

— Label

— Button
— TextField
— ImageView
— conteneurs graphiques

Tous ces éléments héritent de la classe `Node`.

14.5 Structure d'une application JavaFX

Une application JavaFX est une classe qui hérite de la classe `Application`.

Cette classe doit redéfinir la méthode :

```
start(Stage stage)
```

Cette méthode est appelée automatiquement au démarrage de l'application.

14.6 Premier programme JavaFX

Le programme suivant crée une fenêtre simple.

```
import javafx.application.Application;
import javafx.stage.Stage;

public class Main extends Application {

    @Override
    public void start(Stage stage) {

        stage.setTitle("Ma première application JavaFX");

        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

14.7 Explication du programme

14.7.1 Héritage de la classe Application

La classe principale hérite de :

`Application`

Cette classe permet de démarrer une application JavaFX.

14.7.2 La méthode start

La méthode :

`start(Stage stage)`

est appelée automatiquement lors du lancement de l'application.
Le paramètre `stage` représente la fenêtre principale.

14.7.3 Afficher la fenêtre

La méthode suivante affiche la fenêtre :

```
stage.show();
```

Sans cet appel, la fenêtre ne serait pas visible.

14.8 Ajouter du contenu à la fenêtre

Pour afficher des éléments graphiques, il faut créer une **Scene**.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.stage.Stage;

public class Main extends Application {

    @Override
    public void start(Stage stage) {

        Label label = new Label("Bonjour JavaFX");

        Scene scene = new Scene(label, 400, 200);

        stage.setTitle("Application JavaFX");
        stage.setScene(scene);
```

```
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

14.9 Analyse du programme

14.9.1 Création d'un composant

```
Label label = new Label("Bonjour JavaFX");
```

Un `Label` est un composant graphique permettant d'afficher du texte.

14.9.2 Création de la scène

```
Scene scene = new Scene(label, 400, 200);
```

Cette instruction crée une scène contenant le label.
Les valeurs :
— 400 : largeur
— 200 : hauteur
correspondent à la taille de la fenêtre.

14.9.3 Association de la scène à la fenêtre

```
stage.setScene(scene);
```

Cette instruction place la scène dans la fenêtre.

14.10 Cycle de vie d'une application JavaFX

Le fonctionnement d'une application JavaFX suit les étapes suivantes :

1. appel de la méthode `main()`
2. appel de la méthode `launch()`
3. création de l'application
4. appel de la méthode `start()`
5. affichage de la fenêtre

Dans la plupart des cas, le développeur travaille principalement dans la méthode :

```
start()
```

14.11 Bonnes pratiques

Lors du développement d'une application JavaFX, il est recommandé de :

— garder des classes simples et lisibles

— séparer l'interface graphique de la logique métier

— éviter de placer toute la logique dans la classe principale

— organiser le projet en plusieurs classes

Nous utiliserons notamment l'architecture **MVC** vue dans les chapitres précédents.

14.12 Exercices

14.12.1 Exercice 1

Créer une application JavaFX qui affiche une fenêtre :

— titre : `Mon application`

— largeur : 500

— hauteur : 300

14.12.2 Exercice 2

Modifier l'application précédente pour afficher un texte :

Bienvenue dans mon application JavaFX

14.12.3 Exercice 3

Créer une application contenant deux labels :

— Nom

— Prénom

Nous verrons dans les chapitres suivants comment ajouter des champs de saisie et des boutons pour rendre l'application interactive.

Chapitre 15

Tests unitaires avec JUnit

15.1 Pourquoi tester un programme

Lorsqu'un programme devient plus complexe, il devient difficile de vérifier manuellement que toutes les fonctionnalités fonctionnent correctement.

Les **tests unitaires** permettent de vérifier automatiquement le comportement d'une partie du programme.

Un test consiste à :

— exécuter une méthode ;

— comparer le résultat obtenu avec le résultat attendu ;

— signaler une erreur si le résultat est incorrect.

Les tests permettent notamment :

— de vérifier qu'une méthode fonctionne correctement ;

— d'éviter les régressions lors de modifications du code ;

— de sécuriser les évolutions d'une application.

15.2 Principe d'un test unitaire

Un test unitaire vérifie le comportement d'une méthode isolée.

Considérons la méthode suivante :

```
1 public int carre(int x) {
2     return x * x;
3 }
```

Un test peut vérifier que cette méthode retourne bien le résultat attendu.

Par exemple :

— `carre(3)` doit retourner `9`

— `carre(5)` doit retourner `25`

Si la méthode retourne une valeur incorrecte, le test doit échouer.

15.3 La bibliothèque JUnit

JUnit est la bibliothèque de test la plus utilisée en Java.

Elle permet :

— d'écrire des tests ;

— d'exécuter les tests automatiquement ;

— de vérifier les résultats obtenus.

Un test est simplement une méthode annotée avec `@Test`.

```
import org.junit.jupiter.api.Test;

public class CalculTest {

    @Test
    void testCarre() {

    }

}
```

15.4 Les assertions

Les assertions permettent de vérifier qu'un résultat correspond à ce qui est attendu.

L'assertion la plus courante est `assertEquals`.

```
import static org.junit.jupiter.api.Assertions.*;

@Test
void testCarre() {

    int resultat = carre(3);

    assertEquals(9, resultat);

}
```

Si la valeur retournée est différente de 9, le test échoue.

Assertions courantes

JUnit propose plusieurs types d'assertions.

```
assertEquals(valeurAttendue, valeurObtenue);
```

```

assertTrue(condition);

assertFalse(condition);

assertNull(objet);

assertNotNull(objet);
```

Ces méthodes permettent de vérifier différents comportements du programme.

15.5 Tester une classe

Prenons une classe simple :

```
public class Livre {

    private String titre;

    public Livre(String titre) {
        this.titre = titre;
    }

    public String getTitre() {
        return titre;
    }

}
```

On peut créer une classe de test associée.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class LivreTest {

    @Test
    void testTitre() {

        Livre livre = new Livre("1984");

        assertEquals("1984", livre.getTitre());

    }
```

```
14
15 }
```

Le test vérifie que le titre retourné correspond à la valeur attendue.

15.6 Initialisation et nettoyage des tests

Dans certains cas, plusieurs tests utilisent les mêmes objets ou nécessitent une préparation particulière.

JUnit propose des méthodes spéciales permettant d'exécuter du code avant ou après les tests.

Ces méthodes sont appelées **méthodes du cycle de vie des tests**.

15.6.1 Méthodes principales

JUnit propose quatre annotations importantes :

- `@BeforeAll`
- `@AfterAll`
- `@BeforeEach`
- `@AfterEach`

@BeforeAll

La méthode annotée avec `@BeforeAll` est exécutée **une seule fois avant tous les tests**.

Elle est généralement utilisée pour effectuer une initialisation globale.

Exemple :

```
1 @BeforeAll
2 static void initialisation() {
3     System.out.println("Debut des tests");
4 }
```

@AfterAll

La méthode annotée avec `@AfterAll` est exécutée **une seule fois après tous les tests**.

Elle permet généralement de libérer des ressources.

```
1 @AfterAll
2 static void nettoyage() {
3     System.out.println("Fin des tests");
4 }
```

@BeforeEach

La méthode annotée avec `@BeforeEach` est exécutée **avant chaque test**.
Elle permet de préparer les objets nécessaires aux tests.

```
@BeforeEach
void setUp() {
    compte = new CompteBancaire("Alice", 100);
}
```

Ainsi, chaque test commence avec un objet propre.

@AfterEach

La méthode annotée avec `@AfterEach` est exécutée **après chaque test**.
Elle est souvent utilisée pour effectuer un nettoyage.

```
@AfterEach
void tearDown() {
    compte = null;
}
```

Résumé du cycle d'exécution

L'ordre d'exécution des méthodes est le suivant :

```
BeforeAll

    BeforeEach
        Test
    AfterEach

    BeforeEach
        Test
    AfterEach

AfterAll
```

15.6.2 Exemple complet

```
import org.junit.jupiter.api.*;

public class CompteBancaireTest {

```

```
5     CompteBancaire compte;
6
7     @BeforeAll
8     static void debutTests() {
9         System.out.println("Debut des tests");
10    }
11
12    @BeforeEach
13    void initialisation() {
14        compte = new CompteBancaire("Alice", 100);
15    }
16
17    @Test
18    void testDepot() {
19        compte.deposer(50);
20        assertEquals(150, compte.getSolde());
21    }
22
23    @Test
24    void testRetrait() {
25        compte.retirer(20);
26        assertEquals(80, compte.getSolde());
27    }
28
29    @AfterEach
30    void nettoyage() {
31        compte = null;
32    }
33
34    @AfterAll
35    static void finTests() {
36        System.out.println("Fin des tests");
37    }
38
39 }
```

15.7 Tester les exceptions

Il est également possible de vérifier qu'une méthode lance une exception.

Exemple :

```
assertThrows(
    IllegalArgumentException.class,
    () -> livre.setTitre("")
);
```

Le test réussit uniquement si l'exception est déclenchée.

15.8 Organisation des tests

Dans un projet Java, les tests sont généralement placés dans un dossier séparé.

Structure classique :

```
src
  main
     java
         Livre.java

  test
      java
          LivreTest.java
```

Chaque classe métier possède généralement une classe de test correspondante.

15.9 Utilisation de JUnit dans Eclipse

L'environnement de développement Eclipse permet d'utiliser JUnit très facilement.

Dans la plupart des cas, il n'est pas nécessaire de télécharger manuellement la bibliothèque.

Eclipse propose un mécanisme automatique pour ajouter JUnit au projet.

Création d'une classe de test

Pour créer une classe de test :

1. faire un clic droit sur une classe de ton projet ;
2. sélectionner **New** ;
3. choisir **JUnit Test Case**.

Une fenêtre de configuration apparaît.

Il suffit alors de :

— choisir la version de JUnit (JUnit 5 est recommandée) ;

— sélectionner la classe à tester ;

— valider la création de la classe.

Eclipse ajoute automatiquement la bibliothèque JUnit au projet.

Structure générée

Eclipse génère automatiquement une classe de test :

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class CalculTest {

    @Test
    void test() {
        fail("Not yet implemented");
    }

}
```

Il suffit ensuite de remplacer ce code par les tests souhaités.

Exécution d'un test

Pour exécuter un test :

1. faire un clic droit sur la classe de test ;
2. sélectionner **Run As** ;
3. choisir **JUnit Test**.

Eclipse affiche alors le résultat des tests dans une vue dédiée.

— vert : tous les tests ont réussi ;

— rouge : au moins un test a échoué.

15.10 Conclusion

Les tests unitaires permettent de vérifier automatiquement le bon fonctionnement d'un programme.

Ils offrent plusieurs avantages :

— détection rapide des erreurs ;

— amélioration de la qualité du code ;

— sécurisation des modifications.

JUnit est l'outil le plus utilisé en Java pour écrire et exécuter ces tests.